

Webová aplikace pro správu a vizualizaci dat z připojených senzorů na platformě Raspberry Pi

Web Application for Managing and Visualising Data from Connected Sensors on
the Raspberry Pi Platform

Martin Řihák

Bakalářská práce

Vedoucí práce: Ing. Martin Radvanský

Ostrava, 2026

Zadání bakalářské práce

Student:

Martin Řihák

Studijní program:

B0613A140014 Informatika

Téma:

Webová aplikace pro správu a vizualizaci dat z připojených senzorů na platformě Raspberry Pi

Web Application for Managing and Visualising Data from Connected Sensors on the Raspberry Pi Platform

Jazyk vypracování:

čeština

Zásady pro vypracování:

Navrhněte webovou aplikaci, která bude sbírat, zpracovávat a vizualizovat data z libovolného počtu připojených senzorů. Výsledné zobrazení hodnot a historie stavů senzorů bude zobrazeno ve formě uživatelsky konfigurovatelného dashboardu. Aplikaci navrhněte takovým způsobem, aby bylo možné připojit libovolný senzor, na základě definice jeho základních vlastností. Při výběru nového senzoru umožněte odvození senzorů z již existujícího. Komunikace se senzory bude pomocí sběrnice RS-485.

Hlavní body zadání:

1. Proveďte srovnání existujících přístupů pro tvorbu webových aplikací na hardwarové platformě Raspberry Pi.
2. Proveďte rešerši existujících aplikací pro správu dat ze senzorů a proveďte jejich porovnání.
3. Seznamte se s principy komunikace s rozhraním RS-485 na Raspberry Pi.
4. Proveďte návrh a implementaci aplikace.
5. Ověřte funkčnost navržené aplikace.
6. Vytvořte uživatelskou dokumentaci k aplikaci.

Seznam doporučené odborné literatury:

- [1] FARLEY, David. Modern Software Engineering: Doing What Works to Build Better Software Faster. Addison-Wesley Professional, 2021.
- [2] INGENO, Joseph. Software Architect's Handbook: Become a successful software architect by implementing effective architecture concepts. Packt Publishing Ltd, 2018.
- [3] ECKERSON, Wayne W. Performance dashboards: measuring, monitoring, and managing your business. John Wiley & Sons, 2010.
- [4] STANDING, Craig. The requirements of methodologies for developing web applications. In: Web Engineering: Principles and Techniques. IGI Global, 2005. p. 261-280.
- [5] GRINBERG, Miguel. Flask web development: developing web applications with python. " O'Reilly Media, Inc.", 2018.
- [6] MACKAY, Steve; WRIGHT, Edwin; PARK, John. Practical Data Communications for Instrumentation and Control. Elsevier, 2003.

Formální náležitosti a rozsah bakalářské práce stanoví pokyny pro vypracování zveřejněné na webových stránkách fakulty.

Vedoucí bakalářské práce: **Ing. Martin Radvanský**

Datum zadání: 01.09.2024

Datum odevzdání: 30.04.2026

Garant studijního programu: Ing. Tomáš Fabián, Ph.D.

V IS EDISON zadáno: 18.08.2025 10:00:30

Abstrakt

Tato Bakalářská práce se zabývá návrhem a implementací webové aplikace pro správu a vizualizaci dat z připojených senzorů na platformě Raspberry Pi. Cílem je poskytnout uživateli konfigurovatelné rozhraní, které mu umožní přidávání nových senzorů, sledování aktuálních hodnot senzorů, jejich historii a stav v reálném čase. Sensory jsou připojeny pomocí komunikační sběrnice RS-485, která umožňuje přenos dat mezi zařízeními.

Klíčová slova

Raspberry Pi; RS-485; Python; SQLite; Monitorování senzorů; Zpracování dat; Webová aplikace

Abstract

This bachelor's thesis focuses on the design and implementation of a web application for managing and visualizing data from connected sensors on the Raspberry Pi platform. The aim is to provide the user with a configurable interface that enables adding new sensors, monitoring their current values, accessing historical data, and tracking their real-time status. The sensors are connected via the RS-485 communication bus, which facilitates data transmission between devices.

Keywords

Raspberry Pi; RS-485; Python; SQLite; Sensor monitoring; Data processing; Web application

Poděkování

Rád bych na tomto místě poděkoval všem, kteří mi s prací pomohli, protože bez nich by tato práce nevznikla.

Mé poděkování patří především vedoucímu bakalářské práce, panu Ing. Martinu Radvanskému, za odborné vedení, cenné rady a připomínky během celého procesu zpracování práce. Rovněž mu děkuji za zapůjčení platformy Raspberry Pi a senzorů, díky nimž bylo možné navrženou aplikaci nejen implementovat, ale i prakticky ověřit její funkčnost.

Obsah

Seznam použitých symbolů a zkratk	10
Seznam obrázků	12
Seznam tabulek	14
1 Úvod	15
2 Jednodeskové počítače	16
2.1 Raspberry Pi	17
2.1.1 Různé verze Raspberry Pi	17
2.1.2 Raspberry Pi 5 Model B	18
2.1.3 Porovnání generací Raspberry Pi	19
2.2 Význam a využití jednodeskových počítačů	20
3 Tvorba webových aplikací	21
3.1 FrontEnd	21
3.1.1 HTML	22
3.1.2 CSS	22
3.1.3 JavaScript	22
3.1.4 React	23
3.1.5 Angular	23
3.1.6 Vue.js	23
3.1.7 Vite	24
3.2 BackEnd	24
3.2.1 Django	24
3.2.2 Spring Boot	25
3.2.3 Express.js	25
3.2.4 Flask	25
3.3 Databáze	26

3.3.1	Lokální uložení	26
3.3.2	Cloud	26
3.4	Srovnání přístupů	27
3.4.1	Srovnání frontendových přístupů	27
3.4.2	Srovnání backendových přístupů	28
3.4.3	Srovnání databázových řešení	29
4	Rešerše existujících aplikací	30
4.1	ThingsBoard	30
4.2	Grafana s InfluxDB	31
4.3	Home Assistant	31
4.4	Node-RED	32
4.5	Srovnání	32
5	Komunikace se RS-485	34
5.1	Architektura a zapojení RS-485 v systému	34
5.2	Využití RS-485 v praxi	35
5.2.1	Výhody a důvody volby RS-485	35
5.3	Komunikační model RS-485	36
5.3.1	Half a Full duplex komunikace	36
5.3.2	Komunikační Protokoly	37
5.4	Možnosti připojení RS-485 k Raspberry Pi	37
5.5	Senzory	38
5.5.1	Senzory přiblížení	38
5.5.2	Světelné senzory	38
5.5.3	Teplotní senzory	38
5.5.4	Senzory kvality vody	39
5.5.5	Senzor vlhkosti	39
6	Praktická část	40
6.1	Požadavky na aplikaci	40
6.1.0.1	Scénář UC2	43
6.1.0.2	Scénář UC3	44
6.2	Návrh systému	45
6.2.1	Návrhové vzory a principy	45
6.2.2	Návrh databáze	46
6.2.3	Návrh komunikace se senzory	49
6.2.4	Návrh bezpečnosti	51
6.2.5	Návrh uživatelského rozhraní	53

6.3	Implementace	54
6.3.1	Nastavení prostředí	54
6.4	Backend	54
6.4.1	ModbusManager	55
6.4.1.1	Komunikace se senzory	55
6.4.1.2	Ukládání dat ze senzorů a mezipaměť	56
6.4.1.3	Plánované úlohy a vlákna na pozadí	57
6.4.2	Uživatelé a autentizace	58
6.4.3	REST API endpointy	59
6.5	Frontend	60
6.5.1	Směrování a ochrana rout	60
6.5.2	Komunikace s backendem	61
6.5.3	Interaktivní dashboard a widgety	62
6.5.4	Správa senzorů	64
6.5.5	Předpověď počasí	66
6.5.6	Export a import dat	68
6.5.7	Administrátorská správa	69
6.5.8	Rozložení a responzivní design	70
6.6	Ověření funkčnosti navržené aplikace	71
6.6.1	Výpadek komunikace	71
6.6.1.1	Výpadek senzoru na sběrnici RS-485	71
6.6.1.2	Výpadek sběrnice RS-485	74
6.6.2	Porovnání naměřených dat	74
6.7	Nasazení aplikace	76
6.7.1	Předpoklady	76
6.7.2	Spuštění systému	76
6.7.3	Vytvoření administrátorského účtu	77
7	Závěr	78
	Literatura	79
	Přílohy	81
A	Přehled REST endpointů	82
B	Struktura systému	85

C	Dokumentace systému	87
C.1	Hlavní Frontend systému	87
C.1.1	Základní popis hlavního dashboardu	87
C.1.2	Detailní popis hlavního dashboardu	88
C.1.3	Rozhraní pro vytvoření nového widgetu	88
C.1.4	Senzory uživatele	89
C.1.5	Rozhraní pro přidání senzoru	89
C.1.6	Formulář pro úpravu senzoru	90
C.1.7	Vyhledávání lokality pro předpověď	90
C.1.8	Výsledky předpovědi a porovnání senzorů	91
C.1.9	Export naměřených dat	91
C.1.10	Import dat do vybraného senzoru	92
C.1.11	Administrátorská stránka	92
C.1.12	Formulář pro přidání nového uživatele	93
C.1.13	Formulář pro registraci uživatele	93
C.1.14	Formulář pro přihlášení uživatele	94

Seznam použitých zkratek a symbolů

API	– Application Programming Interface
DB	– Database
IoT	– Internet of Things
UI	– User Interface
UX	– User Experience
URL	– Uniform Resource Locator
HTTP	– Hypertext Transfer Protocol
JSON	– JavaScript Object Notation
CSV	– Comma-Separated Values
RPi	– Raspberry Pi
GPIO	– General Purpose Input/Output
RS-485	– Recommended Standard 485
UART	– Universal Asynchronous Receiver-Transmitter
ORM	– Object-Relational Mapping
MVC	– Model-View-Controller
REST	– Representational State Transfer
SQL	– Structured Query Language
CSS	– Cascading Style Sheets
HTML	– Hypertext Markup Language
Py	– Python
JS	– JavaScript
AJAX	– Asynchronous JavaScript and XML
GUI	– Graphical User Interface
RAM	– Random Access Memory
CPU	– Central Processing Unit
GPU	– Graphics Processing Unit
LAN	– Local Area Network
WAN	– Wide Area Network

SSH	– Secure Shell
SDK	– Software Development Kit
CLI	– Command Line Interface
VNC	– Virtual Network Computing
RISC	– Reduced Instruction Set Computer

Seznam obrázků

2.1	RaspBerry Pi 5 (4)	18
3.1	Architektura webové aplikace	22
5.1	RS-485 Modul pro Raspberry Pi(29)	34
5.2	Minimální úroveň signálu(28)	36
5.3	Full-Duplex a Half-Duplex struktura (28)	36
5.4	Rozhraní RS-485 DB9	37
5.5	Senzor teploty a vlhkosti SHT20 s RS485	39
6.1	Use Case Diagram	42
6.2	ER diagram databáze	47
6.3	Doménový model	48
6.4	Sekvenční diagram čtení dat ze senzoru přes Modbus RTU	50
6.5	Sekvenční diagram JWT autentizace a obnovy tokenu	52
6.6	Hlavní Dashboard	63
6.7	Potvrzovací dialog pro smazání dashboardu	63
6.8	Teplotní Widget typu value	64
6.9	Formulář na úpravu senzoru	65
6.10	Vyhledávání lokality s našeptávačem a výběr časového rozsahu	66
6.11	Předpověď počasí pro konkrétní období v zadané lokalitě	67
6.12	Porovnání hodinové teploty z Open-Meteo s hodnotami senzoru systému a denních karet	67
6.13	Export dat z senzorů	68
6.14	Import dat do vybraného senzoru	69
6.15	Stránka po správu uživatelů	70
6.16	Widget typu value s deaktivovaným senzorem a zprávou „Senzor je neaktivní“	72
6.17	Grafový widget s aktivním senzorem bez dostupných dat	72
6.18	Stránka při výpadku komunikace se serverem	73

6.19	Zobrazení chybové zprávy	74
6.20	Chybová hláška komponenty ModbusStatusBanner při odpojení sběrnice RS-485 . .	74
6.21	Graf s porovnáním dat z externí API s daty z teplotního senzoru	75
6.22	Graf s porovnáním dat z externí API s daty z vlhkostního senzoru	75
6.23	Diagram nasazení systému	76

Seznam tabulek

2.1	Srovnání Raspberry Pi 5 s předchozími generacemi(5)	19
3.1	Srovnání frontendových přístupů	27
3.2	Srovnání backendových frameworků	28
3.3	Srovnání databázových řešení	29
4.1	Detailní srovnání analyzovaných IoT platforem	33
6.1	Přehled uživatelských rolí a jejich oprávnění	51
6.2	Přehled navržených rout frontendové aplikace	53
6.3	Přehled Flask Blueprintů a jejich URL prefixů	55
6.4	Přehled klíčových API volání modulu apiService	62
A.1	Přehled REST endpointů	84

Kapitola 1

Úvod

V dnešním světě technologie hrají klíčovou roli v mnoha aspektech našeho života, mohou nám sloužit k vykonávání práce, k zábavě nebo také k usnadnění denních aktivit. Sbírání, zpracovávání a vizualizace dat se stává nezbytnou součástí moderního světa. Efektivní práce s daty umožňuje organizacím optimalizovat procesy, snižovat náklady a přijímat lepší rozhodnutí na základě přesných a aktuálních informací.

Systémy založené na propojených senzorech nacházejí využití v celé řadě oborů. V průmyslu jsou využívány k monitorování výrobních linek, prediktivní údržbě strojů a optimalizaci spotřeby energie. V domácí automatizaci zajišťují inteligentní řízení osvětlení, vytápění a bezpečnostních systémů. Ve zdravotnictví pomáhají sledovat životní funkce pacientů nebo monitorovat hygienické podmínky v nemocnicích.

Klíčovou součástí tohoto systému je jednodeskový počítač Raspberry Pi, který je schopen komunikovat s různými typy senzorů. Raspberry Pi je ideální volbou díky své flexibilitě, cenové dostupnosti a široké komunitě jeho uživatelů. Díky sběrnici RS-485 bude zajišťovat komunikaci s připojenými senzory. Naměřená data budou zpracována v reálném čase a ukládána do lokální databáze s technologií SQLite, která je efektivní pro malé a jednoduché řešení a poskytuje nenáročnou správu dat. Data budou uživatelům zprostředkována pomocí webové aplikace, která nabídne přehlednou vizualizaci aktuálních i historických dat. Aplikace bude také umožňovat uživatelům přidávat nové senzory do systému.

Cílem této práce je navrhnout a implementovat webovou aplikaci, která umožní správu a vizualizaci dat z připojených senzorů prostřednictvím platformy Raspberry Pi. Aplikace poskytne uživatelsky konfigurovatelné rozhraní, umožní sběr a ukládání dat do lokální databáze, vizualizaci aktuálních i historických hodnot a správu připojených zařízení. Důraz je kladen na flexibilitu, jednoduchou rozšiřitelnost a možnost přizpůsobení různým typům senzorů. První část práce je teoretická, kde jsou popsány použité technologie a hardware. Druhá část se věnuje praktické realizaci celého řešení.

Kapitola 2

Jednodeskové počítače

Jednodeskové počítače (anglicky Single-Board Computers, SBC) jsou kompaktní počítače, které integrují všechny základní komponenty – jako procesor, paměť, vstupně/výstupní rozhraní a napájení na jediné desce plošných spojů. Tyto počítače často disponují rozhraními, jako jsou USB porty, grafické výstupy a síťová připojení, což umožňuje jejich využití v různých aplikacích, od vzdělávání po průmyslové nasazení.

Jako hlavní procesor se obvykle používají RISC nebo ARM procesory, které kombinují vysokou efektivitu a nízkou spotřebu energie, což je ideální pro vestavné systémy. Operační systémy, jako je Linux, Raspbian nebo specializované distribuce (například Ubuntu Core), poskytují širokou škálu funkcí a flexibilitu. Novější přístroje mají 1 až 8 GB RAM a několik GB paměti flash, která slouží jako náhrada pevného disku. Pokud počítač nedisponuje integrovanou flash pamětí, je možné použít SD kartu nebo USB zařízení pro ukládání dat a operačního systému.

Typické vybavení zahrnuje jeden až dva USB porty, grafický výstup (například HDMI) a další vstupně/výstupní zařízení. Některé modely nabízejí RJ-45 port pro pevné připojení k síti, zatímco modernější varianty často disponují i Wi-Fi a Bluetooth pro bezdrátovou konektivitu. Díky těmto vlastnostem mohou jednodeskové počítače částečně nahradit tradiční stolní počítače a umožňují připojení myši, klávesnice nebo monitoru.

V případech, kdy nejsou dostupné porty pro připojení vstupně/výstupních zařízení, lze k ovládní jednodeskového počítače využít vzdálený přístup prostřednictvím služeb jako VNC nebo SSH, které umožňují spravovat zařízení po síti z jiného počítače. Tato flexibilita činí jednodeskové počítače ideálními pro široké spektrum aplikací, včetně IoT (Internetu věcí), domácí automatizace, robotiky a vzdělávacích projektů.(1)

2.1 Raspberry Pi

Raspberry Pi je řada cenově dostupných jednodeskových počítačů vyvinutých britskou nadací Raspberry Pi Foundation s cílem podpořit výuku informatiky a programování. Tyto kompaktní zařízení, velikostí srovnatelné s kreditní kartou, nabízejí plnohodnotné funkce osobního počítače, včetně USB portů, grafických výstupů a síťových připojení. Díky své univerzálnosti a nízké ceně našly uplatnění nejen ve vzdělávání, ale i v průmyslových aplikacích, domácí automatizaci a projektech IoT. (2)

Od svého uvedení v roce 2012 prošly modely Raspberry Pi několika generacemi, které přinesly zvýšení výkonu, kapacity paměti a rozšíření konektivity. Například model Raspberry Pi 4 B nabízí až 8 GB RAM, gigabitový Ethernet a podporu pro duální 4K displeje, což jej činí vhodným pro náročnější aplikace. (2)

Raspberry Pi podporuje různé operační systémy, přičemž oficiálním a nejpoužívanějším je Raspberry Pi OS, založený na Debianu. Tato flexibilita umožňuje uživatelům přizpůsobit zařízení specifickým potřebám, ať už jde o serverové nasazení, multimediální centrum nebo vývojové prostředí pro programování. (3)

Díky aktivní komunitě a rozsáhlé dokumentaci je Raspberry Pi ideální platformou pro začátečníky i pokročilé uživatele, kteří chtějí experimentovat s hardwarem a softwarem, vytvářet prototypy nebo se učit novým technologiím. (2)

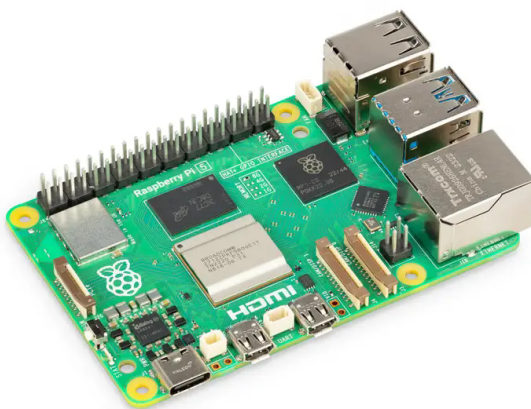
2.1.1 Různé verze Raspberry Pi

- **Raspberry Pi Model A** - Raspberry Pi Model A byl prvním modelem této řady a nabízel základní funkce pro méně náročné projekty. Obsahoval 256 MB RAM, jeden USB port a neobsahoval ethernetový port. Tento model byl ideální pro aplikace, kde nebyla potřeba síťová konektivita a kladl se důraz na nízkou spotřebu energie.
- **Raspberry Pi Model B** - Raspberry Pi Model B přinesl větší univerzálnost díky přítomnosti ethernetového portu a zvýšené kapacitě RAM na 512 MB. Nabízel také dva USB porty, což umožnilo připojení více periferií. Model B byl určen pro projekty, které vyžadují síťovou konektivitu a větší výkon.
- **Raspberry Pi 2 Model B** - Výrazně vylepšený model oproti předchozím verzím, obsahující 1 GB RAM, čtyři USB porty a ethernetové připojení. Díky čtyřjádrovému procesoru byl schopen zvládat náročnější úlohy.
- **Raspberry Pi 3 Model B** - Tento model přinesl integrované Wi-Fi a Bluetooth, což z něj učinilo ideální volbu pro bezdrátové aplikace. Obsahoval 1 GB RAM, čtyři USB porty a ethernetový port, což jej činilo univerzálním nástrojem pro širokou škálu projektů.

- **Raspberry Pi Zero** - Ultrakompaktní model s nízkou cenou a omezeným počtem I/O portů. Verze *Zero W* zahrnuje Wi-Fi a Bluetooth, což jej činí ideálním pro minimalistické projekty s nízkou spotřebou energie.
- **Raspberry Pi 400** - Tento model integruje Raspberry Pi přímo do klávesnice, což z něj činí kompletní stolní počítač. Nabízí výkon Modelu 4 B a je určen především pro vzdělávací účely.

2.1.2 Raspberry Pi 5 Model B

Raspberry Pi 5 je nejnovější verzí populárního jednodeskového počítače, která přináší významná vylepšení oproti předchozím generacím. Je vybaven čtyřjádrovým procesorem ARM Cortex-A76 s frekvencí 2,4 GHz, což zajišťuje výrazné zvýšení výkonu. Dále nabízí až 8 GB LPDDR4X RAM, což umožňuje efektivní multitasking a provoz náročnějších aplikací. Zařízení podporuje duální 4K displeje prostřednictvím dvou micro-HDMI portů, což je ideální pro uživatele vyžadující vysoké rozlišení. Mezi další vylepšení patří rychlejší USB 3.0 porty, gigabitový Ethernet a podpora pro PCIe 2.0, což umožňuje připojení vysokorychlostních úložišť. Tyto inovace činí Raspberry Pi 5 výkonnějším a univerzálnějším nástrojem pro širokou škálu projektů. Tuto verzi vidíme na obrázku 2.1



Obrázek 2.1: RaspBerry Pi 5 (4)

2.1.3 Porovnání generací Raspberry Pi

V tabulce 2.1 vidíme porovnání a specifikace popisovaných alternativ.

Specifikace	Raspberry Pi 5	Raspberry Pi 4 Model 8GB	Raspberry Pi 3 Model
SoC	Broadcom BCM2712	Broadcom BCM2711	Broadcom BCM2837B0
Datum vydání	Říjen 2023	Květen 2020	Březen 2018
Výrobní proces	16nm	28nm	40nm
Počet jader CPU	4× ARM Cortex-A76	4× ARM Cortex-A72	4× ARM Cortex-A53
Frekvence CPU	2,4 GHz	1,5 GHz	1,4 GHz
Podpora paměti	8GB LPDDR4X až 176GB/s	8GB LPDDR4 2400MHz	1GB LPDDR2
Cache	64K L1/jádro, 512K L2/jádro, 2MB L3 sdílená	80K L1/jádro, 1MB L2 sdílená	80K L1/jádro, 1MB L2 sdílená
Grafický procesor	12-jádrový VideoCore VII @ 800MHz	VideoCore VI @ 500MHz	VideoCore IV @ 400MHz
Cena (MSRP)	7600 Kč	4200 Kč	900Kč

Tabulka 2.1: Srovnání Raspberry Pi 5 s předchozími generacemi(5)

2.2 Význam a využití jednodeskových počítačů

Jednodeskové počítače (SBC) jsou klíčovou technologií díky své nízké ceně, kompaktnosti a univerzálnosti. Umožňují implementaci široké škály projektů v následujících oblastech.

- **Internet věcí (IoT)** - SBC jsou hojně využívány pro monitorování a řízení chytrých zařízení, sběr dat ze senzorů a automatizaci procesů, což z nich činí základní stavební kámen IoT systémů.
- **Vzdělávání** - Díky nízké ceně a přívětivému ekosystému jsou jednodeskové počítače oblíbené pro výuku programování, elektroniky a práce s hardwarem. Raspberry Pi se stal symbolem pro popularizaci technologií mezi studenty a začátečníky.
- **Průmyslové aplikace** - SBC najdou uplatnění v průmyslu při monitorování výroby, prediktivní údržbě a řízení robotických systémů, a to díky své flexibilitě a schopnosti integrovat různá rozhraní.
- **Domácí automatizace** - Používají se k řízení osvětlení, vytápění, bezpečnostních systémů a dalších prvků chytré domácnosti. Díky své nenáročnosti a přizpůsobitelnosti umožňují jednoduchou automatizaci domácích procesů.
- **Multimediální centra** - SBC jsou ideální pro použití jako streamovací zařízení, přehrávače videí nebo servery pro domácí média díky své schopnosti zpracovávat multimédia a snadno se integrovat do existujících systémů.
- **Prototypování a vývoj** - SBC jsou díky své nízké ceně a snadné dostupnosti vhodné pro rychlé testování a implementaci nových konceptů, a to jak ve výzkumu, tak v komerčním vývoji.

Kapitola 3

Tvorba webových aplikací

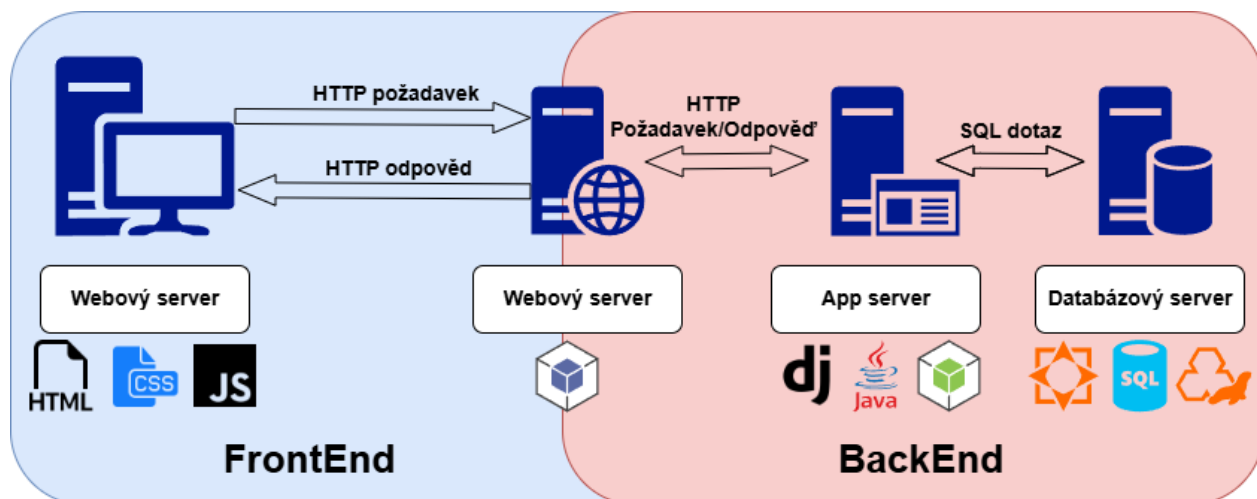
Tvorba webových aplikací zahrnuje návrh, vývoj a implementaci, které fungují prostřednictvím webového prohlížeče. Tyto aplikace jsou přístupné přes internet nebo lokální síť. Jejich popularita neustále roste díky své univerzálnosti a snadné dostupnosti. Webové aplikace jsou využívány v mnoha oblastech, od jednoduchých nástrojů pro správu dat až po komplexní systémy zahrnující e-commerce, IoT nebo umělou inteligenci.

Vývoj webových aplikací je typicky rozdělen do tří hlavních částí: frontend, který tvoří uživatelské rozhraní; backend, kde probíhá logika a zpracování dat; a databázová vrstva, která spravuje ukládání a načítání dat. Tento přístup umožňuje efektivní správu, škálovatelnost a údržbu aplikací.

Vývoj webových aplikací je typicky rozdělen do tří vrstev. První vrstvou je frontend neboli prezentační vrstva. Ta zajišťuje interakci s uživatelem a prezentaci dat. Druhá vrstva, backend, neboli aplikační vrstva, obsahuje logiku a zpracování požadavků od klienta. Zahrnuje servery, které spravují data potřebná k fungování aplikace. Třetí vrstva je datová struktura, která se stará o ukládání a načítání dat na serveru. Tento přístup umožňuje efektivní správu, škálovatelnost a údržbu aplikací.

3.1 FrontEnd

Frontend je viditelná část webové aplikace, se kterou uživatel přímo interaguje. Zahrnuje vizuální a interaktivní prvky, jako jsou rozvržení stránky, navigační menu, tlačítka a formuláře. Cílem vývoje frontendu je vytvořit uživatelsky přívětivé, responzivní a vizuálně atraktivní prostředí. Frontend je postaven na technologiích jako HTML (pro strukturu), CSS (pro vzhled) a JavaScript (pro interaktivitu). Moderní frameworky, jako jsou React, Angular a Vue.js, umožňují efektivnější vývoj složitých rozhraní. Na obrázku 3.1 vidíme jednoduchý diagram, který zobrazuje komunikaci mezi klientem, tedy webovým prohlížečem, přes který uživatel komunikuje s webovým serverem a posílá a přijímá HTTP zprávy.⁽⁶⁾



Obrázek 3.1: Architektura webové aplikace

3.1.1 HTML

Hypertext Markup Language nebo HTML označuje v informatice název značkovacího jazyka používaného pro tvorbu webových stránek, které jsou propojeny hypertextovými odkazy. HTML je hlavním z jazyků pro vytváření stránek v systému World Wide Web, který umožňuje publikaci. Jazyk HTML je charakterizován množinou značek (tzv. tagů) a jejich vlastnostmi (atributy) definovanými pro danou verzi. Tento zápis byl převzat ze SGML a dále upravován v závislosti na aktuálních trendech Internetu. Aktuální verzí jazyka je verze 5.0, která přinesla po patnáctileté odmlce nové funkcionality a možnosti vývoje webových stránek.(7)

3.1.2 CSS

CSS neboli kaskádové styly jsou v informatice jazykem pro popis způsobu zobrazení elementů na stránkách napsaných v jazycích HTML, XHTML nebo XML. Jazyk byl navržen standardizační organizací W3C. Hlavním smyslem je umožnit návrhářům oddělit vzhled dokumentu od jeho struktury a obsahu.(8)

3.1.3 JavaScript

JavaScript (zkráceně JS) je odlehčený interpretovaný (resp. just-in-time kompilovaný) programovací jazyk s podporou first-class funkcí, který je standardizován pod specifikací ECMA-262 (ECMAScript). Jedná se o objektově orientovaný, prototypově založený, multiparadigmatický jazyk podporující imperativní, funkcionální i objektový styl programování. Spolu s HTML a CSS tvoří JavaScript jednu ze tří základních technologií webu a je dnes využíván pro klientskou interaktivitu na drtivé většině webových stránek. Ačkoli je JavaScript nejznámější jako skriptovací jazyk webo-

vých prohlížečů, používá se i v non-browser prostředích, zejména v běhovém prostředí Node.js, které umožňuje nasazení JS na straně serveru. (9)

3.1.4 React

React je open-source JavaScriptová knihovna pro deklarativní tvorbu uživatelských rozhraní, kterou vyvíjí společnost Meta od roku 2013 a která je zveřejněna pod licencí MIT. Klíčovým principem je komponentová architektura – uživatelské rozhraní se skládá z malých, znovupoužitelných komponent, zapsaných jako JavaScriptové funkce, které využívají rozšířenou syntaxi JSX, jež kombinuje JavaScript s HTML-podobnou notací. Pro efektivní aktualizaci uživatelského rozhraní React využívá tzv. virtuální DOM, díky němuž překresluje pouze ty části stránky, které se reálně změnily. React prosazuje jednosměrný tok dat – data proudí dolů přes props a události nahoru přes callback funkce, což zjednodušuje předvídatelnost stavových změn a ladění. Mezi hlavní výhody patří rozsáhlý ekosystém (Next.js, React Router, Redux, Material UI) a transferovatelnost dovedností do tvorby mobilních aplikací prostřednictvím React Native. V kombinaci s vizualizačními knihovnami jako Recharts, Chart.js nebo Plotly je React typickou volbou pro tvorbu interaktivních dashboardů. (10, 11)

3.1.5 Angular

Angular je open-source frontendový framework vyvíjený společností Google od roku 2016, zveřejněný pod licencí MIT. Jedná se o kompletní přepis předchůdce AngularJS (2010) v jazyce TypeScript, jehož použití Angular striktně vyžaduje. Angular se prezentuje jako platforma s principem *batteries included* – obsahuje vlastní router, HTTP klient, formulářový systém, animace a CLI v jednom konzistentním balíku. Přináší striktní strukturu kódu, která je výhodná pro velké enterprise aplikace a týmy. Pro tvorbu dashboardů a vizualizací IoT dat existují integrace s knihovnami jako ngx-charts nebo Highcharts. Typickým nasazením jsou rozsáhlé podnikové SPA a administrátorské konzole. (12)

3.1.6 Vue.js

Vue.js je open-source progresivní JavaScriptový framework pro tvorbu uživatelských rozhraní. Označení „progresivní“ vyjadřuje inkrementální adoptovatelnost – Vue lze začlenit jako jednoduchý `<script>` tag do existující stránky, nebo jej postupně rozšiřovat na plnohodnotný framework s routerem (Vue Router) a state managementem (Pinia). Klíčovými technickými vlastnostmi jsou reaktivní data-binding založený na ES Proxy (Vue 3), single-file components kombinující `<template>`, `<script>` a `<style>` v jednom souboru a kompilátorem optimalizovaný virtual DOM. Vue.js je typickým řešením pro MVPs, microcity i rozsáhlé SPA, kde se cení rychlost vývoje a flexibilita. (13)

3.1.7 Vite

Vite je moderní frontendový build tool, jehož cílem je poskytnout rychlou a štíhlou vývojářskou zkušenost pro moderní webové projekty. Skládá se ze dvou hlavních částí: dev serveru využívajícího nativní ES moduly s podporou Hot Module Replacement (HMR) a build příkazu produkujícího optimalizovaný produkční bundle. Hlavní výhodou je téměř okamžitý start vývojového serveru a milisekundové reflektování změn v prohlížeči. Vite je dnes doporučovaným způsobem vytvoření React aplikace a nabízí předkonfigurované šablony pro React, Vue, Svelte i TypeScript.(14)

3.2 BackEnd

Backend, česky serverová část představuje část webové aplikace, která není přímo viditelná pro uživatele. Zahrnuje server, databázi a aplikační logiku, které společně zajišťují funkčnost aplikace. Backend zpracovává požadavky od uživatele, komunikuje s databází, provádí potřebné operace a vrací odpovědi zpět na frontend, kde jsou prezentovány uživateli. Mezi hlavní úkoly backendu patří správa dat, autentizace a autorizace uživatelů, zajištění bezpečnosti a integrace s dalšími službami. Pro vývoj backendu se využívají různé programovací jazyky, jako jsou PHP, Python, Ruby, Java nebo Node.js, a frameworky, které usnadňují tvorbu robustních a škálovatelných aplikací.

3.2.1 Django

Django je framework pro vývoj webových aplikací v Pythonu, zaměřený na rychlý vývoj a čistý kód. Vestavěné nástroje, jako jsou ORM, autentizace a administrační rozhraní, jej činí vhodným pro rozsáhlé aplikace vyžadující spolehlivost, bezpečnost a rychlé nasazení. Často se využívá v e-commerce, CMS a datově náročných aplikacích, jako jsou sociální sítě. Django je ideální pro vývoj robustních aplikací s mnoha vestavěnými funkcemi, zejména tam, kde je kladen důraz na bezpečnost. Mezi hlavní výhody patří ORM, autentizace, administrační rozhraní, ochrana proti útokům (XSS, CSRF, SQL Injection) a aktivní komunita. Nevýhodami jsou strmá křivka učení, vyšší nároky na výkon a možná nadbytečná robustnost pro menší aplikace.(15)

3.2.2 Spring Boot

Spring Boot je framework pro tvorbu podnikových aplikací v Javě, který usnadňuje konfiguraci a vývoj. Je postavený na Spring Frameworku, který umožňuje rychlé vytvoření produkčních aplikací s minimální konfigurací. Podporuje mikroslužby, což z něj činí oblíbenou volbu ve velkých podnicích. Díky široké nabídce modulů a integrací je vhodný pro aplikace s robustním backendovým řešením.

Spring Boot se využívá v podnikových systémech, mikroslužbách a distribuovaných systémech. Hlavní výhody zahrnují snadnou konfiguraci, rychlý vývoj a škálovatelnost. Nevýhody zahrnují vyšší systémové nároky a složitější konfiguraci pro začátečníky.(16)

3.2.3 Express.js

Express.js je rychlý, minimalistický a tzv. neopinionovaný webový framework pro běhové prostředí Node.js. V současnosti je považován za de facto standardní serverový framework Node.js ekosystému a tvoří backendovou složku populárních stacků MERN (MongoDB, Express, React, Node) a MEAN (MongoDB, Express, Angular, Node). Nabízí robustní routing, middleware architekturu a nástroje pro tvorbu webových aplikací a REST API, aniž by vynucoval konkrétní ORM nebo strukturu projektu. Díky neblokujícímu I/O modelu prostředí Node.js dosahuje vysokého výkonu pro real-time scénáře a je vhodný i pro nasazení na výpočetně omezených zařízeních, jako je Raspberry Pi.(17)

3.2.4 Flask

Flask je lehký a flexibilní webový framework pro Python, který umožňuje rychlý vývoj webových aplikací. Je klasifikován jako microframework, protože neobsahuje vestavěné komponenty, jako jsou databázové ORM nebo validaci formulářů, což vývojářům dává volnost použít pouze to, co potřebují. Díky své jednoduchosti je ideální pro menší projekty, prototypy a výukové účely. Nabízí snadné rozšíření pomocí rozšiřujících balíčků pro autentizaci, práci s databází nebo API. Jeho přehledná struktura pomáhá udržet kód čistý a srozumitelný. I přes svou jednoduchost je dostatečně výkonný pro reálné aplikace a používají ho i velké firmy.(18)

3.3 Databáze

Databáze organizuje a uchovává data, což umožňuje efektivní správu a přístup k informacím. Ve webových aplikacích slouží k ukládání uživatelských dat, obsahu a transakčních záznamů. Systémy řízení databází (DBMS) umožňují manipulaci s daty, dotazování a zajištění integrity. V prostředí IoT jsou databáze klíčové pro zpracování velkých objemů dat generovaných senzory. Data musí být spolehlivě uložena pro analýzu a vizualizaci v reálném čase. Moderní databázové systémy nabízejí filtrování, třídění a rychlé vyhledávání.

Pro IoT se využívají lokální databáze běžící na zařízení i cloudová řešení, která zajišťují centralizované ukládání a vzdálený přístup. Výběr technologie závisí na požadavcích systému.(19)

3.3.1 Lokální uložení

Lokální úložiště je systém, který ukládá data přímo na zařízení, kde jsou generována, bez potřeby připojení k internetu nebo vzdálenému serveru. V případě Raspberry Pi, které často funguje jako lokální server, se data ukládají přímo na Micro SD kartu.

Mezi nejvíce používané databázové systémy na Raspberry Pi patří MySQL, MariaDB a PostgreSQL, které jsou vhodné pro rozsáhlejší aplikace vyžadující robustní databázová řešení. Nicméně pro menší projekty nebo aplikace s nízkými nároky na výkon se doporučuje použít SQLite.

SQLite je relační databázový systém, který nevyžaduje samostatný server a ukládá celou databázi do jednoho souboru. Díky tomu je ideální volbou pro prostředí s omezenými zdroji, jako je právě Raspberry Pi. Proto zajišťuje rychlý a bezproblémový vývoj aplikací bez nutnosti složité konfigurace databázového serveru.

3.3.2 Cloud

Cloudové databáze jsou databázové systémy, které umožňují ukládání dat na vzdálených serverech, což zajišťuje vysokou dostupnost a škálovatelnost odkudkoliv. Tento přístup je ideální pro aplikace, které potřebují agregovat data z více zařízení nebo umožňují přístup více uživatelům. Navíc oproti lokálnímu úložišti poskytují cloudové služby často pokročilé analytické nástroje a automatické zálohování dat. V prostředí IoT se cloudové databáze používají například pro monitorovací systémy a analytické platformy.

Na trhu existuje spousta různých řešení od firem, které se snaží tyto služby provozovat a poskytovat uživatelům. Záleží tedy pouze na potřebách každého uživatele, protože ani jedno nelze považovat za nejlepší. Pro práci na malých projektech, které vypracovávají jedinci, je nejlepší využít bezplatné služby, kde si lze jednotlivá řešení vyzkoušet. Naopak u rozsáhlejších projektů je rozumné sáhnout po placených verzích, které uživatele neomezuji a přinášejí spoustu benefitů.

3.4 Srovnání přístupů

Tato kapitola shrnuje silné a slabé stránky technologií popsaných v předchozích sekcích a hodnotí jejich vhodnost pro aktuální projekt – webovou aplikaci běžící na Raspberry Pi. Srovnání je rozděleno do tří částí podle vrstev aplikace – frontend, backend a databáze.

3.4.1 Srovnání frontendových přístupů

Vanilla JS, tedy čistý JavaScript bez frameworku, se hodí pro statické weby a drobné widgety, ale s rostoucí velikostí aplikace klesá jeho udržitelnost. Pro interaktivní dashboardy s real-time daty ze senzorů je tedy nepraktický. Angular je oproti tomu komplexní framework určený pro velké enterprise aplikace, kde se cení pevná struktura a typová bezpečnost. Pro menší IoT projekty však bývá zbytečně robustní. Vue.js je vyváženým kompromisem – jeho reaktivní data-binding usnadňuje tvorbu dashboardů s automatickou aktualizací hodnot ze senzorů, což ho činí vhodným pro malé a střední IoT projekty. React je vhodný pro single-page aplikace a dynamická uživatelská rozhraní, jako jsou právě interaktivní dashboardy pro vizualizaci IoT dat, a to díky knihovně `react-grid-layout`. V kombinaci s knihovnami `Recharts`, `Chart.js` nebo `Plotly` nabízí robustní řešení pro tvorbu grafů a real-time vizualizací. Podle Stack Overflow Developer Survey 2025 jej využívá 44,7 % vývojářů, čímž je nejpopulárnějším frontendovým řešením.

Pro aktuální projekt byl zvolen React, protože se hodí pro tvorbu dynamického dashboardu pro IoT data a má největší komunitu a ekosystém, což usnadňuje hledání řešení a dokumentace. Volba frontendového frameworku má na Raspberry Pi minimální vliv, protože build se provádí na vývojářském stroji a Pi pouze servíruje statické soubory klientovi.(11, 20)

Kritérium	Vanilla JS	React	Angular	Vue.js
Velikost bundlu (gzip)	0 KB	cca 45 KB	cca 48–135 KB	cca 33 KB
Typ DOM	Přímý DOM	Virtual DOM	Incremental DOM	Virtual DOM
Jazyk	JavaScript	JavaScript / JSX	TypeScript	JavaScript / HTML šablony
Hlavní podporovatel	ECMA / W3C	Meta	Google	Evan You + komunita
Vydáno (rok)	1995	2013	2016	2014
Vhodnost pro projekt	Střední	Vysoká	Nízká	Vysoká

Tabulka 3.1: Srovnání frontendových přístupů

3.4.2 Srovnání backendových přístupů

Django je framework s přístupem *batteries included*, který se hodí pro CMS, e-commerce a datově orientované aplikace s důrazem na bezpečnost a rychlý vývoj. Spring Boot je naopak enterprise řešení v Javě vhodné pro podnikové systémy, mikroslužby a aplikace s vysokou zátěží, dosahuje nejvyššího výkonu ze všech srovnávaných frameworků. Jeho nevýhodou je vysoká paměťová náročnost JVM, která se pohybuje kolem 125–150 MB pouze pro framework. Express.js je minimalistický framework v Node.js ekosystému, který je vhodný pro real-time aplikace, REST API a mikroslužby. Díky neblokujícímu I/O modelu nabízí dobrý výkon i na slabším hardwaru. Flask je microframework v Pythonu s nejnižší paměťovou náročností, který se hodí pro malé až střední projekty, REST API a IoT aplikace. Jeho jednoduchá struktura a velký ekosystém rozšíření umožňují použít pouze to, co je skutečně potřeba.

Pro aktuální projekt byl zvolen Flask, protože nabízí nejnižší paměťovou náročnost ze všech srovnávaných frameworků a Python dokáže komunikovat díky knihovně `minimalmodbus` se sběrnici RS-485, což bylo klíčovým důvodem pro jeho volbu místo Express.js. Spring Boot byl pro Raspberry Pi vyloučen kvůli vysoké paměťové náročnosti JVM a Django je pro jednoduchou IoT aplikaci zbytečně robustní.(21)

Kritérium	Flask	Django	Express.js	Spring Boot
Jazyk	Python	Python	JavaScript	Java
Vydáno (rok)	2010	2005	2010	2014
Výkon	nejnižší	nízký	střední	nejvyšší
Min. paměť pro běh	cca 14–16 MB	cca 18–19 MB	cca 20–40 MB	cca 125–150 MB
Vestavěný ORM	Ne	Ano (Django ORM)	Ne	Ano (JPA / Hibernate)
Vestavěný admin panel	Ne	Ano	Ne	Ne
GitHub stars (2025)	cca 71,4 tis.	cca 82,5 tis.	cca 68,9 tis.	cca 80,5 tis.
Vhodnost pro projekt	Vysoká	Střední	Vysoká	Nízká

Tabulka 3.2: Srovnání backendových frameworků

3.4.3 Srovnání databázových řešení

SQLite se hodí pro embedded a IoT aplikace, lokální data logging a malé webové projekty. MySQL a MariaDB jsou vhodné pro víceuživatelské weby a aplikace vyžadující paralelní přístup. PostgreSQL je vhodný pro komplexní aplikace s pokročilými datovými strukturami a geo-daty. InfluxDB je specializovaná time-series databáze určená pro vysokofrekvenční zápis časových dat ze senzorů a monitorovacích systémů. Cloudové databáze (např. AWS RDS, Azure SQL, Google Cloud SQL) jsou určeny pro distribuované systémy s mnoha uzly a vysokými nároky na škálovatelnost. (22)

Pro aktuální projekt byl zvolen SQLite, protože se jedná o IoT aplikaci běžící na jednom zařízení (Raspberry Pi) s lokálním ukládáním na microSD kartu. SQLite nevyžaduje samostatný databázový server ani konfiguraci, má nejnižší paměťovou stopu a poskytuje dostatečný výkon pro všechny scénáře projektu. InfluxDB by byla vhodná pro ukládání časových řad ze senzorů, ale pro rozsah tohoto projektu představuje zbytečně komplexní řešení s vyššími nároky na paměť a údržbu. Cloudové řešení nebylo zvoleno kvůli závislosti na internetovém připojení, síťové latenci a měsíčním poplatkům.

Kritérium	SQLite	MySQL	MariaDB	Postgre	InfluxDB	Cloud
Typ databáze	Relační	Relační	Relační	Objekt-relační	Time-series	Relační
Architektura	Embedded	Klient-server	Klient-server	Klient-server	Klient-server	Managed cloud
Velikost instalace	cca 590 KB	cca 200–250 MB	cca 225–265 MB	cca 250 MB	cca 50–80 MB	—
Min. RAM pro běh	cca 4 MB	cca 7,5–128 MB	cca 7,5–128 MB	cca 8 MB+	cca 100 MB+	—
Konkurenční zápis	1 zapisovatel	Plný	Plný	Plný (MVCC)	Plný	Plný
Závislost na internetu	Ne	Ne	Ne	Ne	Ne	Ano
Síťová latence	0 ms	0 ms	0 ms	0 ms	0 ms	20–200 ms
Licence	Public Domain	GPLv2	GPLv2	PostgreSQL MIT Lic.	MIT	Pay-per-use
Vhodnost pro projekt	Vysoká	Střední	Střední	Nízká	Střední	Nízká

Tabulka 3.3: Srovnání databázových řešení

Kapitola 4

Rešerše existujících aplikací

Tato práce má za úkol vytvořit systém, který bude sbírat data ze senzorů a zobrazovat je uživateli pomocí webové aplikace. Tento způsob již dlouhá léta existuje i v praxi a mnoho firem vytvořilo svá řešení, jak tento úkol efektivně realizovat. S rostoucí dostupností senzorových zařízení a jejich integrací do oblastí, jako je průmysl, zdravotnictví nebo zemědělství, se čím dál více objevují nová a inovativní řešení v oblasti správy dat.

Účelem této kapitoly je provést podrobnou analýzu existujících aplikací a platforem, které slouží k podobnému účelu, a následně je porovnat z hlediska funkcionality, flexibility a dalších klíčových parametrů. Výsledky této rešerše pomohou lépe definovat požadavky na navrhovaný systém a poskytnou inspiraci pro jeho implementaci.

V rámci rešerše budou vybrány čtyři existující aplikace, které reprezentují rozličné přístupy k řešení tohoto problému. U každé aplikace bude uveden podrobný popis jejího fungování, klíčových vlastností a použitých technologií. Následně bude provedeno jejich srovnání na základě předem definovaných kritérií.

4.1 ThingsBoard

ThingsBoard je open-source platforma určená pro správu zařízení v oblasti Internetu věcí (IoT). Jádro je postaveno na technologii Java a frameworku Spring Boot s mikroservisní architekturou, kde jednotlivé transportní moduly pro protokoly MQTT, CoAP, HTTP, LwM2M a SNMP komunikují s jádrem prostřednictvím sběrnice Apache Kafka. Pro perzistenci dat platforma využívá databázi PostgreSQL, případně v kombinaci s Apache Cassandra nebo TimescaleDB pro zpracování velkých objemů telemetrie.

Klíčovým prvkem je vizuální Rule Engine založený na principu řetězců uzlů (rule chains), který umožňuje definovat zpracování přicházejících zpráv, jejich filtrování, transformaci, agregaci a generování alarmů. Skripty v uzlech lze psát v jazyce JavaScript nebo v interním ThingsBoard Expression Language (TBEL). Nad daty platforma poskytuje knihovnu více než 600 konfigurovatelných wi-

dgetů a flexibilní dashboardy. ThingsBoard je distribuován ve dvou edicích: Community Edition se základní funkcionalitou zdarma a Professional Edition, která přidává single sign-on, white-labeling a rozšířené integrace (23).

4.2 Grafana s InfluxDB

Kombinace databáze InfluxDB a vizualizačního nástroje Grafana je modulární stack pro sběr, ukládání a zobrazování dat ze senzorů. InfluxDB je specializovaná databáze časových řad napsaná v jazyce Go, optimalizovaná pro rychlý zápis i dotazování velkých objemů časově orientovaných dat. Data jsou organizována do tzv. bucketů a dotazována prostřednictvím dotazovacích jazyků InfluxQL nebo Flux. Grafana je analytická platforma napsaná v jazycích Go a TypeScript, která se k InfluxDB připojuje jako k datovému zdroji a poskytuje konfigurovatelné dashboardy s grafy, tabulkami, indikátory a systémem upozornění.

Typická architektura na Raspberry Pi sestává ze tří nezávisle běžících komponent: brokeru MQTT (např. Mosquitto), který přijímá data od senzorů, agenta Telegraf nebo vlastního skriptu, jenž data zapisuje do InfluxDB, a Grafany jako vizualizační vrstvy. Obě hlavní komponenty jsou distribuovány pod licencemi MIT (InfluxDB) a AGPL v3 (Grafana) a poskytují oficiální balíčky pro architekturu ARM, takže je lze nainstalovat přímo na Raspberry Pi OS (24).

4.3 Home Assistant

Home Assistant je open-source platforma pro domácí automatizaci a vizualizaci dat ze senzorů, kterou v roce 2013 založil Paulus Schoutsen. Projekt je distribuován pod licencí Apache 2.0 a od roku 2024 jej zajišťuje nezisková organizace Open Home Foundation. Jádro je napsáno v jazyce Python s plnou podporou knihovny asyncio. Platforma podporuje čtyři instalační varianty (Home Assistant OS, Container, Supervised a Core), z nichž Home Assistant OS představuje doporučenou plug-and-play distribuci pro Raspberry Pi.

Platforma obsahuje téměř 3 000 vestavěných integrací pokrývajících protokoly MQTT, Zigbee, Z-Wave, Matter, Bluetooth, KNX a Modbus včetně sběrnice RS-485. Pro perzistenci dat slouží integrace Recorder s podporou databází SQLite, MariaDB a PostgreSQL přes vrstvu SQLAlchemy; pro dlouhodobé ukládání časových řad lze využít integraci s InfluxDB. Vizualizační vrstva Dashboard je postavena na webových komponentách (TypeScript, Lit) a nabízí konfigurovatelné karty editovatelné jak vizuálně, tak deklarativně v jazyce YAML. Mobilní aplikace pro Android a iOS poskytuje přístup k dashboardům i ze zařízení uživatele (25).

4.4 Node-RED

Node-RED je open-source nástroj pro vizuální programování datových toků (tzv. flow-based programming), který původně vyvinula společnost IBM v roce 2013 a od roku 2016 je součástí nadace OpenJS Foundation pod licencí Apache 2.0. Jedná se o jednu z mála aplikací oficiálně předinstalovaných v doporučeném obrazu Raspberry Pi OS. Nástroj je postaven na běhovém prostředí Node.js a v prohlížeči poskytuje editor, ve kterém uživatel skládá aplikační logiku přetahováním uzlů (nodes) a jejich propojováním do toků (flows).

Knihovna komunitních uzlů obsahuje stovky integrací pokrývajících protokoly MQTT, HTTP, WebSocket, Modbus včetně sběrnice RS-485, sériovou komunikaci a přímý přístup ke GPIO pinům Raspberry Pi. Doplněk node-red-dashboard přidává sadu uzlů pro tvorbu jednoduchého webového rozhraní s grafy, indikátory, tlačítka a formuláři, generovaného automaticky z editoru bez nutnosti psát kód v jazyce HTML či JavaScript. Vestavěný dashboard nepodporuje vícero uživatelů ani jejich role a Node-RED neposkytuje vlastní úložiště dat, takže pro dlouhodobé ukládání naměřených hodnot je nutné jej propojit s externí databází (26).

4.5 Srovnání

Každé z popsaných řešení cílí na jiný typ projektů, byť všechna pokrývají úlohu sběru a vizualizace senzorických dat. Pro objektivní srovnání byla využita metodika z recenzovaného přehledu komerčních a open-source IoT platforem pro smart city aplikace (27), který hodnotí platformy podle protokolové podpory, škálovatelnosti, otevřenosti a celkové vhodnosti.

ThingsBoard se uplatňuje v projektech s větším počtem zařízení a požadavkem na centralizovanou správu, jako je průmyslové monitorování, chytrá města, environmentální monitoring nebo řízení spotřeby energie. Grafana s InfluxDB je vhodná pro projekty zaměřené primárně na vizualizaci a analýzu časových řad, kde je hlavním cílem dlouhodobý monitoring velkých objemů senzorických dat a detekce odchylek. Home Assistant je nejvhodnější pro domácí automatizaci a menší IoT instalace s důrazem na širokou podporu chytrých zařízení a jednoduchost konfigurace. Node-RED se uplatní pro rychlé prototypování a propojování existujících služeb a senzorů, zejména v projektech, kde nejsou nárokovány pokročilé funkce typu správy uživatelů či perzistence dat.

Žádné z popsaných řešení proto není univerzální a výběr výsledné platformy závisí na konkrétních požadavcích projektu. Detailní porovnání jednotlivých řešení podle technologických a funkčních kritérií je zobrazeno v tabulce 4.1.

Kritérium	ThingsBoard	Grafana + InfluxDB	Home Assistant	Node-RED
Typ licence	Apache 2.0	MIT (InfluxDB), AGPL v3 (Grafana)	Apache 2.0	Apache 2.0
Cena	CE zdarma; PE od 10 USD/měs.	Plně zdarma	Plně zdarma	Plně zdarma
Technologie	Java, Spring Boot, Apache Kafka	Go, TypeScript	Python, asyncio	Node.js, JavaScript
Protokoly	MQTT, CoAP, HTTP, LwM2M, SNMP	HTTP, MQTT (přes Telegraf), zápis přes API	MQTT, Zigbee, Z-Wave, Modbus, RS-485	MQTT, HTTP, WebSocket, Modbus, RS-485, GPIO
Datové úložiště	PostgreSQL, Cassandra, TimescaleDB	InfluxDB	SQLite, MariaDB, PostgreSQL, InfluxDB	Závislé na externí DB
Vizualizace	600+ widgetů, SCADA symboly	Pokročilé dashboards, grafy, alerty	Dashboard (karty, sekce)	node-red-dashboard
Podpora RS Pi	Ano (min. 4 GB RAM)	Ano (oficiální ARM balíčky)	Ano (oficiální HA OS image)	Ano (předinstalován v Raspberry Pi OS)
Multi-tenancy	Ano (RBAC v PE)	Ano (Organizations, RBAC)	Ano (role admin/uživatel)	Ne (single-user)
Mobilní aplikace	Ano (Flutter)	Pouze responzivní web	Ano (Android, iOS)	Pouze responzivní web
Složitost nasazení	Střední	Střední (3 komponenty)	Nízká	Velmi nízká
Hlavní výhody	Robustní rule engine, multi-tenancy, široké protokolové pokrytí	Vysoký výkon u časových řad, modální architektura	Velký integrační ekosystém, snadná instalace	Rychlé prototypování, vizuální editor, nízké HW nároky
Hlavní nevýhody	Vyšší HW nároky, složitější konfigurace	Nutnost propojit tři nezávislé komponenty	Méně vhodný pro průmyslové nasazení	Žádné vestavěné úložiště, single-user dashboard

Tabulka 4.1: Detailní srovnání analyzovaných IoT platforem

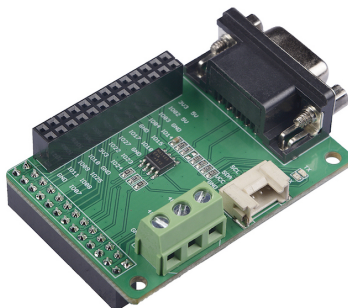
Kapitola 5

Komunikace se RS-485

V rámci této práce je klíčovou součástí komunikace mezi senzory a platformou Raspberry Pi, která probíhá prostřednictvím sběrnice RS-485. Tato sběrnice je široce využívána v průmyslových a automatizačních aplikacích díky své schopnosti umožnit spolehlivý přenos dat na dlouhé vzdálenosti a podporu více zařízení na jedné lince. Cílem je, se seznámit s principy komunikace po sběrnici RS-485, popsat její vlastnosti, způsoby implementace na platformě Raspberry Pi.

5.1 Architektura a zapojení RS-485 v systému

Architektura systému je navržena modulárně, což znamená, že umožňuje snadné rozšiřování přidáváním dalších senzorů. V rámci tohoto systému hraje klíčovou roli Raspberry Pi 4, které slouží jako řídicí jednotka (master), zatímco senzory fungují jako podřízená zařízení (slaves). Díky RS-485 sběrnici 5.1 je možné propojit více senzorů na jedinou komunikační linku, čímž se výrazně snižuje počet vodičů a zvyšuje efektivita celého systému. Systém využívá sběrniceovou topologii Daisy Chain, kde jsou všechna zařízení propojena do jedné linie.(28)



Obrázek 5.1: RS-485 Modul pro Raspberry Pi(29)

5.2 Využití RS-485 v praxi

RS-485 nachází široké uplatnění v různých oblastech díky své schopnosti poskytovat spolehlivý a robustní přenos dat na dlouhé vzdálenosti. Níže jsou uvedeny hlavní oblasti, kde se tento standard běžně využívá:

- **Průmyslová automatizace** – RS-485 se často používá pro propojení PLC, senzorů a akčních členů v průmyslových řídicích systémech.
- **Energetika** – měřicí přístroje, jako jsou elektroměry a analyzátory sítě, často komunikují prostřednictvím RS-485.
- **Dopravní systémy** – využití v železničních a silničních dopravních systémech, například pro řízení semaforů a informačních panelů.
- **Automatizace budov** – řízení HVAC systémů, osvětlení a bezpečnostních zařízení v rámci inteligentních budov.
- **Měřicí a laboratorní technika** – mnoho laboratorních přístrojů používá RS-485 k přenosu dat do počítačů nebo jiných systémů.
- **Lékařská technika** – RS-485 se využívá v nemocničních a diagnostických přístrojích pro spolehlivou datovou komunikaci.

5.2.1 Výhody a důvody volby RS-485

1. **Podpora více zařízení** – umožňuje připojit až 32 zařízení na jednu sběrnici bez potřeby dalších zesilovačů signálu.
2. **Odolnost vůči rušení** – díky diferenciálnímu přenosu signálu je méně náchylná na elektromagnetické rušení.
3. **Dlouhý dosah přenosu** – umožňuje přenos dat až na 1200 metrů.
4. **Efektivní využití vodičů** – využívá pouze dva vodiče (A/B) pro přenos dat a společnou zem (GND).
5. **Nízké náklady na implementaci** - oproti CAN nebo Ethernet.
6. **Spolehlivost v průmyslovém prostředí** – díky robustnímu přenosu a schopnosti fungovat v drsných podmínkách(28)

5.3 Komunikační model RS-485

Sběrnice RS-485 využívá diferenciální přenos, což znamená, že informace se přenášejí jako rozdíl napětí mezi vodiči A (D+) a B (D-). Tento rozdíl je minimálně 1,5 V při zátěži 54 Ω , přičemž standardní přijímače detekují rozdílový vstup již od 200 mV. Jak je znázorněno na obrázku 5.2, tento rozdíl poskytuje dostatečnou rezervu pro spolehlivý přenos dat i při degradaci signálu způsobené délkou kabelu nebo kvalitou konektorů. Tato robustnost umožňuje spolehlivý provoz v náročných průmyslových podmínkách a dělá z RS-485 ideální volbu pro komunikaci na delší vzdálenosti v hlučném prostředí. Díky své odolnosti vůči elektromagnetickému rušení má výhodu oproti jednovodičovým komunikačním standardům.(28)



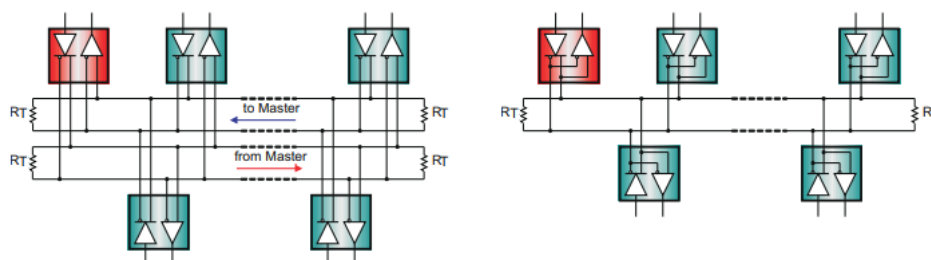
Obrázek 5.2: Minimální úroveň signálu(28)

5.3.1 Half a Full duplex komunikace

Sběrnice RS-485 typicky pracuje v half-duplexním režimu. V tomto režimu se používá pouze jeden signálový pár, což vyžaduje, aby k řízení a příjmu dat docházelo v různých časech.

Plně duplexní implementace naopak vyžaduje dva signálové páry (čtyři vodiče) a plně duplexní transceivery se samostatnými sběrniceovými přístupovými linkami pro vysílač a přijímač. Full-duplex umožňuje uzlu současně přenášet data na jednom uzlu při přijímání dat na druhém páru.

Obě implementace vyžadují řízenou činnost všech uzlů pomocí signálů řízení směru, jako třeba že řidič/přijímač povolí signály, aby bylo zajištěno, že na sběrnici bude vždy aktivní pouze jeden řidič. Pokud více než jeden řidič přistupuje ke sběrnici ve stejnou dobu, vede ke sporu, kterému je potřeba se vždy vyhnout pomocí softwarové kontroly. Schéma vidíme na obrázku 5.3(28)



Obrázek 5.3: Full-Duplex a Half-Duplex struktura (28)

5.3.2 Komunikační Protokoly

Aby sběrnice mohla poskytovat odolnost vůči rušení a schopnost komunikovat na dlouhé vzdálenosti, musí na fyzické vrstvě používat správné komunikační protokoly, které definují pravidla pro výměnu dat mezi zařízeními. Níže uvádím některé z nich.

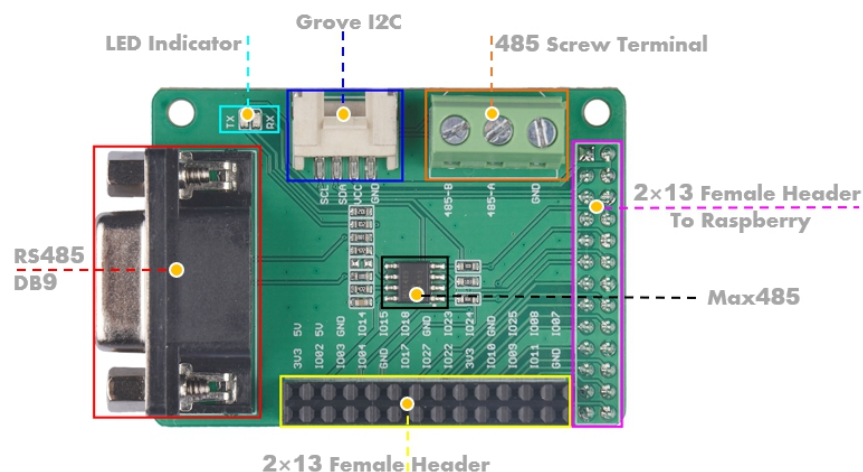
- **Modbus RTU** - U PLC automatů. Využívá jednoduchý master-slave model, kde zařízení master iniciuje komunikaci a zařízení slave odpovídají. Data jsou přenášena v binárním formátu.
- **BACnet MS/TP** - Určený pro automatizaci budov. Komunikace mezi systémy HVAC, osvětlením a bezpečnostními systémy. Mechanismus předávání tokenu pro řízení přístupu k sběrnici
- **Vlastní binární protokoly** - Mnoho výrobců vyvíjí své protokoly pro komunikaci, aby mohli vyhovět přímo jejich specifikacím a požadavkům.

5.4 Možnosti připojení RS-485 k Raspberry Pi

Raspberry Pi samo o sobě nemá nativní RS-485 rozhraní, proto je nutné použít převodník. Na trhu jich můžeme najít spoustu a od různých výrobců. Je tedy na uživateli a na jeho požadavcích, který si vybere. Nejčastěji se používají 2 typy připojení.

- **USB-to-RS-485 adaptér** - který poskytuje jednoduché připojení skrze USB.
- **UART-to-RS-485** - využívá UART port Raspberry Pi, zapojuje se přímo na GPIO piny.

Připojení přes GPIO je obecně vhodnější, pokud chceš přímou kontrolu nad signály pro řízení směru přenosu. Popis jednotlivých portů a konektorů na 5.4



Obrázek 5.4: Rozhraní RS-485 DB9

5.5 Senzory

Senzory hrají důležitou roli v automatizaci jakékoli aplikace tím, že měří a zpracovávají shromážděná data pro detekci změn ve fyzických věcech. Kdykoli dojde ke změně jakéhokoli fyzického stavu, pro který je senzor vyroben, vytvoří měřitelnou odezvu. Existují různé typy senzorů, které se mohou pohybovat od velmi jednoduchých až po složité. Klasifikace senzorů může být založena na jejich specifikacích, jejich převodní metodě, typu použitého materiálu, jeho fyzikálním jevu snímání, vlastnostech, které měří, a oblasti použití. V průmyslových aplikacích jsou senzory často připojovány ke sběrnici RS-485, která díky své odolnosti vůči rušení a možnosti komunikace na delší vzdálenosti umožňuje propojit větší počet senzorů do jedné měřicí sítě. (30)

5.5.1 Senzory přiblížení

Senzory přiblížení detekují objekty bez fyzického kontaktu pomocí elektromagnetického záření, jako je infračervené světlo. Existuje několik typů, včetně indukčních, kapacitních, ultrazvukových, fotoelektrických a magnetických senzorů, přizpůsobených různým aplikacím. Používají se v průmyslu pro detekci objektů, počítání položek, polohování, měření směru pohybu či v parkovacích systémech. Díky své spolehlivosti nacházejí uplatnění v mnoha odvětvích, zejména tam, kde je důležitá bezpečnost a efektivita.(30)

5.5.2 Světelné senzory

Světelné senzory měří intenzitu osvětlení v jednotkách lux a umožňují automatickou regulaci osvětlení podle okolních podmínek. Mohou například vypnout světlo, pokud v místnosti není dostatečný pohyb, nebo upravit jas podle denní doby. Pro vzdálené nasazení v budovách či sklenících jsou dostupné varianty s rozhraním RS-485, které přenášejí naměřené hodnoty na centrální řídicí jednotku přes Modbus RTU a umožňují tak sběr dat z většího počtu měřicích bodů jediným kabelem. Tyto senzory přispívají ke zvýšení pohodlí, úspoře energie a snížení nákladů na elektřinu. (30)

5.5.3 Teplotní senzory

Teplotní senzory monitorují teplotu prostředí a umožňují chytrým termostatům optimalizovat vytápění či chlazení. Pro průmyslový sběr dat jsou nejčastěji nabízeny ve variantách s rozhraním RS-485 (např. čidla PT100, PT1000 či SHT20) 5.5, které komunikují přes protokol Modbus RTU a lze je řetězit po jediné sběrnici na vzdálenosti až 1 200 metrů. Tato vlastnost je činí vhodnými pro nasazení v rozlehlých objektech, sklenících či skladech, kde je třeba měřit teplotu na více místech současně. (30)

5.5.4 Senzory kvality vody

Senzory kvality vody se používají k monitorování různých parametrů, jako je teplota, pH, zákal, vodivost a obsah rozpuštěného kyslíku. Moderní nízkonákladové systémy integrují tyto senzory do platforem IoT pro automatizované a vzdálené řízení kvality vody ve vodohospodářství, akvakultuře nebo úpravárnách vody. (30)

5.5.5 Senzor vlhkosti

Senzor vlhkosti SHT20, jako na obrázku 5.5 se využívá pro měření relativní vlhkosti vzduchu i půdy v automatizovaných systémech. Senzor obsahuje kapacitní měřicí prvek, jehož kapacita se mění v závislosti na obsahu vlhkosti v okolním prostředí. Ve variantě s rozhraním RS-485 je naměřená hodnota digitálně přenášena přes protokol Modbus RTU, což umožňuje spolehlivé čtení dat i na vzdálenosti řádu stovek metrů a připojení většího počtu senzorů na jedinou sběrnici. (30)



Obrázek 5.5: Senzor teploty a vlhkosti SHT20 s RS485

Kapitola 6

Praktická část

Tato kapitola popisuje návrh a realizaci systému dle inženýrských postupů, díky nimž bylo možné vytvořit efektivní a dlouhodobě udržitelný systém, který je snadno škálovatelný.

Řešení, které navrhuji, se zaměřuje na integraci senzorů od různých výrobců do jednoho funkčního systému. Tento systém bude schopen nejen sbírat data ze senzorů, ale také je ukládat, exportovat a vizualizovat. Místo závislosti na externích cloudových službách bude využito lokální úložiště a zpracování dat pomocí jednodeskového počítače Raspberry Pi 4.

Pro sběr dat byly používány dva specifické senzory. Prvním je senzor SHT20, který měří vlhkost a teplotu, a druhým je RS-GZ-N01, zaměřený na měření intenzity světla. Díky této kombinaci je systém schopen poskytnout komplexní přehled o vnitřním prostředí chytré domácnosti.

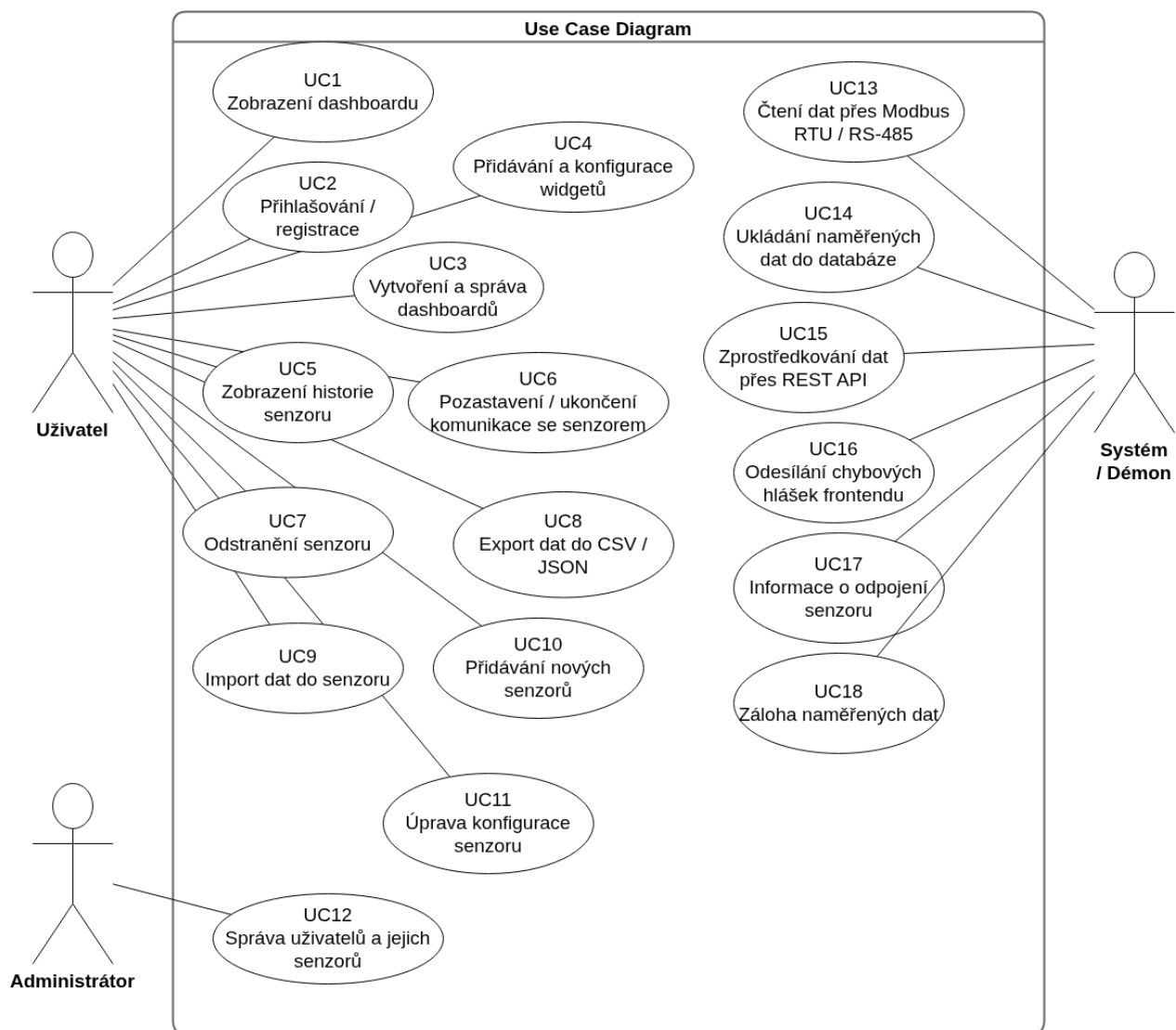
Pro ukládání naměřených dat je použita databáze SQLite, která slouží i pro webovou část aplikace – správu uživatelů, konfiguraci senzorů a dashboardů. Pro backend webové aplikace byl zvolen framework Flask, který díky své jednoduchosti a dostačujícím možnostem vyhovuje potřebám projektu. Komunikace mezi frontendem a backendem probíhá prostřednictvím REST API ve formátu JSON. Celkově systém nabídne uživateli přehledné a intuitivní prostředí, kde bude moci snadno přidávat nové dashboardy nebo senzory, konfigurovat jejich parametry a monitorovat aktuální i historická data ve formě grafů a dalších vizualizací.

6.1 Požadavky na aplikaci

Toto jsou základní případy užití pro navrhovaný systém. Na obrázku 6.1 se nachází celkový návrh use-case diagramu, který znázorňuje interakci uživatelů se systémem.

Každý use case popisuje konkrétní funkcionalitu systému, kterou mohou využít uživatelé nebo administrátoři. Níže jsou detailně popsány nejdůležitější případy užití a také diagram 6.1

- **UC1** – Zobrazuje uživatelem nastavený dashboard s aktuálními daty z navolených senzorů a vybraných elementů.
- **UC2** – Umožňuje přihlašování a registraci uživatelů, kteří mohou systém ovládat.
- **UC3** – Umožňuje vytváření nových dashboardů a jejich správu.
- **UC4** – Dokáže přidávat nové widgety do dashboardu, přesouvat je a měnit jejich velikost.
- **UC5** – Možnost zobrazit historii naměřených dat a základní informace u vybraného senzoru.
- **UC6** – Možnost pozastavit nebo ukončit komunikaci s daným senzorem.
- **UC7** – Odstranit daný senzor ze systému, jak dočasně, tak trvale.
- **UC8** – Export dat vybraného senzoru ve vybraném časovém intervalu do formátu CSV nebo JSON.
- **UC9** – Import dat ze souboru do vybraného senzoru v systému.
- **UC10** – Možnost přidávat nové senzory do systému.
- **UC11** – Umožňuje úpravu konfigurace existujícího senzoru, jako je název, jednotka nebo popis.
- **UC12** – Administrátor může spravovat uživatele a jejich senzory pomocí samostatného UI.
- **UC13** – Démon čte data ze senzorů prostřednictvím protokolu Modbus RTU přes rozhraní RS-485.
- **UC14** – Systém po časovém intervalu ukládá naměřená data do databáze.
- **UC15** – Systém zprostředkovává data pomocí REST API frontendu.
- **UC16** – Systém odesílá chybové hlášky frontendu, který je zobrazuje pomocí alertů.
- **UC17** – Systém informuje uživatele, že byl senzor odpojen, nebo že je nefunkční komunikace.
- **UC18** – Naměřená data se zálohují po nastaveném intervalu do databáze systému.



Obrázek 6.1: Use Case Diagram

6.1.0.1 Scénář UC2

Popis scénáře: Přihlášení uživatele do systému **Aktéři:** Uživatel, Systém **Podmínky pro spuštění:** Uživatel není přihlášen do systému. **Podmínky pro ukončení:** Uživatel je přihlášen a přesměrován na hlavní stránku.

1. Uživatel otevře přihlašovací stránku.
2. Systém zkontroluje přihlašovací stav uživatele.
 - 2.1. Systém zjistí, že uživatel je již přihlášen.
 - 2.2. Systém přesměruje uživatele na hlavní stránku a scénář končí.
3. Systém zobrazí přihlašovací formulář.
4. Uživatel vyplní své uživatelské jméno a heslo.
5. Uživatel klikne na tlačítko "Přihlásit se".
6. Systém provede kontrolu vyplnění formuláře.
 - 7.1. Systém zjistí, že nejsou vyplněna všechna povinná pole.
 - 7.2. Systém zobrazí chybovou hlášku o nevyplněných polích a vrací se na krok 3.
7. Systém odešle přihlašovací údaje na server.
8. Server ověří existenci uživatele s daným uživatelským jménem.
 - 9.1. Server zjistí, že uživatel s daným jménem neexistuje.
 - 9.2. Server vrátí chybovou zprávu o nesprávných přihlašovacích údajích.
 - 9.3. Systém zobrazí chybovou hlášku a vrací se na krok 3.
9. Server ověří správnost hesla.
 - 10.1. Server zjistí, že heslo je nesprávné.
 - 10.2. Systém zobrazí chybovou hlášku a vrací se na krok 3.
10. Server vytvoří JWT token pro přihlášeného uživatele.
11. Server aktualizuje čas posledního přihlášení uživatele.
12. Server vrátí údaje o uživateli a JWT token a uloží ho do lokálního úložiště.
13. Systém zobrazí hlášku o úspěšném přihlášení a přesměruje na hlavní stránku.

6.1.0.2 Scénář UC3

Popis scénáře: Přidání nového widgetu do dashboardu **Aktéři:** Uživatel, Systém **Podmínky pro spuštění:** Uživatel musí být přihlášen do systému. **Podmínky pro ukončení:** Nový widget bude přidán do dashboardu a zobrazen uživateli.

1. Uživatel klikne na tlačítko Vytvořit nový widget v hlavičce dashboardu.
2. Systém zobrazí modální okno s formulářem pro přidání nového widgetu.
3. Uživatel vybere senzor z dropdown menu.
4. Uživatel vybere typ grafu z vizuálního selektoru.
5. Uživatel potvrdí výběr kliknutím na tlačítko Přidat widget.
6. Systém provede kontrolu vyplnění formuláře.
 - 6.1. Systém zjistí, že uživatel nevybral senzor nebo typ grafu.
 - 6.2. Systém zobrazí chybovou hlášku o chybném vyplnění formuláře a vrací se na krok 2.
7. Systém vytvoří nový widget s vybraným senzorem a typem grafu.
8. Systém přidá nový widget do dashboardu.
9. Systém uloží změny do databáze.
10. Systém zavře modální okno a aktualizuje zobrazení dashboardu s novým widgetem.

6.2 Návrh systému

Tato kapitola popisuje návrh systému a jeho prvky. Jsou zde zdokumentována klíčová architektonická rozhodnutí, použité návrhové vzory, datový model a bezpečnostní model aplikace. Cílem návrhu bylo vytvořit systém, který je modulární, snadno rozšiřitelný a provozovatelný na hardware omezené platformě Raspberry Pi, a lze jej používat zcela lokálně a bez nutnosti připojení k internetu.

Systém funguje jako třívrstvá klient-server architektura. Na nejnižší vrstvě se nacházejí fyzické senzory připojené prostřednictvím sběrnice RS-485. Střední vrstvu tvoří Flask backend, který komunikuje se senzory pomocí protokolu Modbus RTU, spravuje uživatele a poskytuje REST API ve formátu JSON. Nejvyšší vrstvou je jednostránková webová aplikace (SPA) implementovaná v knihovně React, která komunikuje s backendem prostřednictvím HTTP požadavků.

Pro ukládání dat je využita databáze SQLite, která slouží jak pro ukládání dat ze senzorů, tak jednotlivých uživatelů a elementů v aplikaci.

6.2.1 Návrhové vzory a principy

Návrh backendu vychází ze tří etablovaných softwarových vzorů, jejichž kombinace zajišťuje přehlednost kódu a nezávislé rozšiřování jednotlivých částí systému.

Blueprint pattern rozděluje backend do samostatných funkčních modulů, z nichž každý obsluhuje konkrétní doménu aplikace (autentizace, senzory, dashboard, widgety, správa uživatelů). Přidání nové funkcionality nevyžaduje zásah do existujícího kódu – stačí vytvořit nový Blueprint a zaregistrovat jej v centrální funkci `init_routes(app)`.

Service layer pattern odděluje aplikační logiku od definice HTTP endpointů. Každý Blueprint pouze přijme požadavek, ověří přihlášení a extrahuje parametry – veškeré zpracování předá příslušné třídě ve vrstvě `services/`.

Repository pattern je realizován nástrojem SQLAlchemy, který mapuje objekty Pythonu na databázové záznamy. Veškerá komunikace s databází probíhá výhradně přes modely definované v adresáři `models/`, nikoliv přímými SQL dotazy, čímž je systém oddělen od konkrétní databázové technologie. Frontend byl navržen jako jednostránková aplikace (SPA). Komponentový přístup knihovny React rozděluje uživatelské rozhraní do znovupoužitelných celků. Komunikace s backendem je centralizována v modulu `apiService.js`, který zapouzdřuje veškerá HTTP volání a zajišťuje jednotné zpracování chybových stavů a obnovu autentizačních tokenů.

6.2.2 Návrh databáze

Pro ukládání dat systému byla zvolena relační databáze SQLite. Tato volba vycházela z požadavků projektu – SQLite nevyžaduje instalaci samostatného databázového serveru a je plně funkční v prostředí Raspberry Pi. Pro potřeby projektu dostačuje jak výkonnostně, tak funkčně.

Databáze obsahuje osm entit rozdělených do tří logických skupin, které odpovídají třem doménám systému: správě uživatelů, správě senzorů a prezentační vrstvě dashboardu. Schéma databáze znázorňuje ER diagram na obrázku 6.2.

Uživatelská vrstva

Uživatelská vrstva zahrnuje tabulku **User**, která uchovává přihlašovací údaje, e-mail a roli uživatele (**admin** nebo **user**). Každý uživatel může vlastnit více dashboardů prostřednictvím relace 1:N a více senzorů prostřednictvím vazební tabulky **UserSensor** (relace M:N). Tato vazba je klíčová pro autorizaci – uživatel má přístup pouze k senzorům, které mu byly přiřazeny.

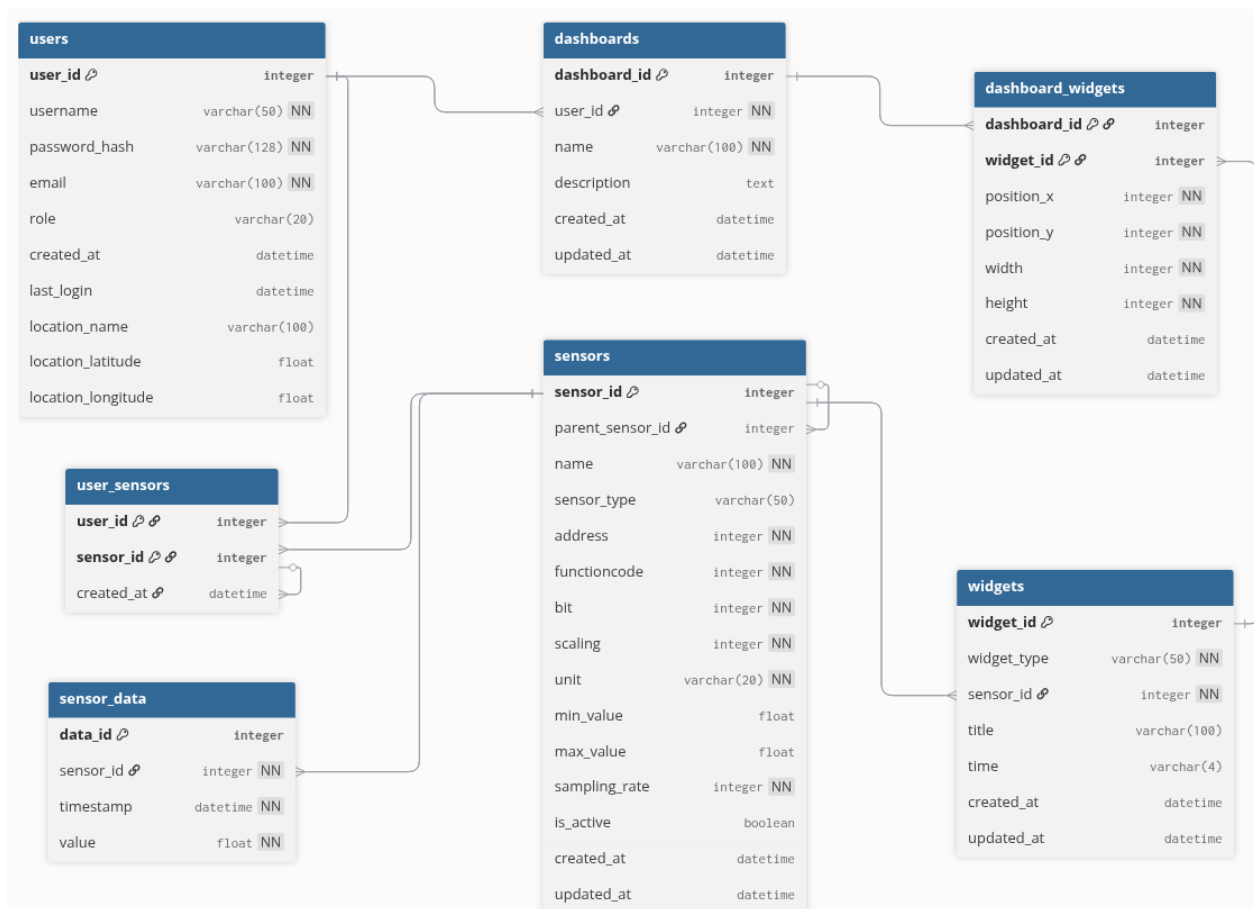
Senzorová vrstva

Senzorová vrstva obsahuje tabulku **Sensor** s metadaty potřebnými pro Modbus komunikaci: adresa registru, funkční kód, baudrate, dělicí konstanta a interval vzorkování. Tabulka **SensorData** ukládá časové řady naměřených hodnot. Na sloupci **timestamp** byl vytvořen databázový index, který výrazně zrychluje dotazy filtrující data za určité časové období – tato operace je nejčastěji prováděna při vykreslování grafů na dashboardu.

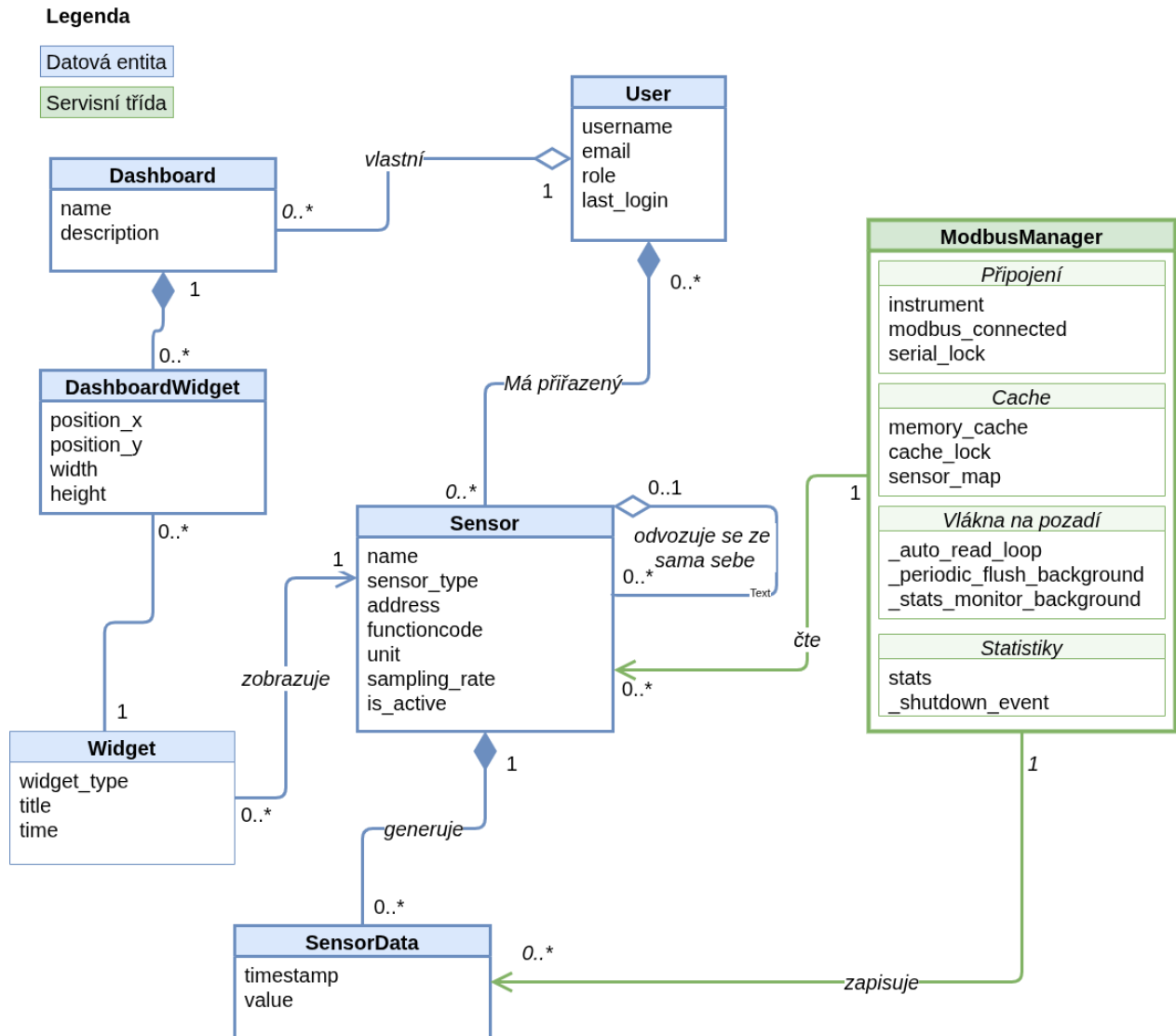
Tabulka **Sensor** dále obsahuje volitelný sloupec **parent_sensor_id**, který umožňuje odvození nového senzoru z již existujícího. Při vytváření odvozeného senzoru jsou parametry komunikace (adresa, funkční kód, baudrate) automaticky předvyplněny z rodičovského senzoru, čímž se zjednodušuje přidávání senzorů stejného typu.

Prezentační vrstva

Prezentační vrstva je tvořena tabulkami **Dashboard** a **Widget** a vazební tabulkou. Tabulka **DashboardWidget** propojuje widgety s dashboardy (relace M:N) a zároveň uchovává pozici a rozměry každého widgetu v mřížce (**position_x**, **position_y**, **width**, **height**). Tabulka **Widget** definuje typ widget a přiřazuje mu konkrétní senzor. Vzájemné vztahy mezi hlavními třídami systému, včetně servisní třídy **ModbusManager**, znázorňuje doménový model na obrázku 6.3.



Obrázek 6.2: ER diagram databáze



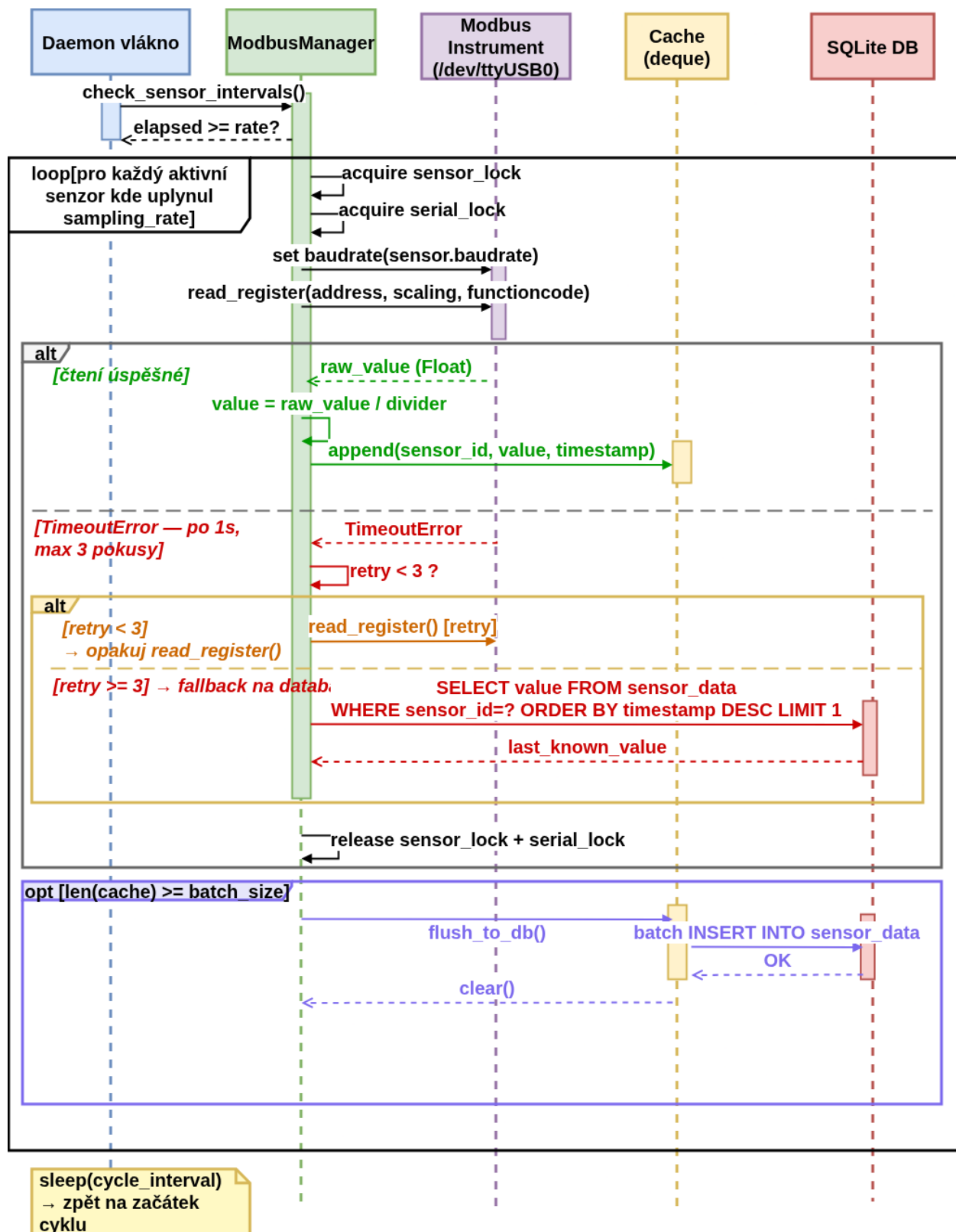
Obrázek 6.3: Doménový model

6.2.3 Návrh komunikace se senzory

Komunikace se senzory připojenými přes sběrnici RS-485 byla navržena ve třídě `ModbusManager`, která zapouzdřuje veškerou logiku sběru dat. Pro přenos dat byl zvolen protokol Modbus RTU realizovaný přes sériový port `/dev/ttyUSB0`.

Sběr dat probíhá ve dvou režimech. Daemon vlákno v pravidelném cyklu kontroluje, zda u každého aktivního senzoru uplynul nastavený interval vzorkování. Pokud ano, provede čtení registru přes Modbus instrument – přičemž přenosová rychlost (baudrate) je nastavena dynamicky před každým čtením. Úspěšně přečtená hodnota je vydělena dělicí konstantou a uložena do paměťové cache typu ring buffer (`deque`). Při dosažení prahové velikosti cache jsou data zapsána dávkově do databáze SQLite, čímž se minimalizuje počet I/O operací na SD kartě.

V případě selhání komunikace (timeout 1 sekunda, maximálně 3 opakované pokusy) systém automaticky načte poslední známou hodnotu z databáze. Tento dvoustupňový fallback mechanismus zajišťuje, že uživatel vždy obdrží data. Celý tok znázorňuje sekvenční diagram na obrázku 6.4.



Obrázek 6.4: Sekvenční diagram čtení dat ze senzoru přes Modbus RTU

6.2.4 Návrh bezpečnosti

Systém využívá dvouúrovňový mechanismus řízení přístupu. První úroveň je založena na JWT tokenech, druhá na uživatelských rolích.

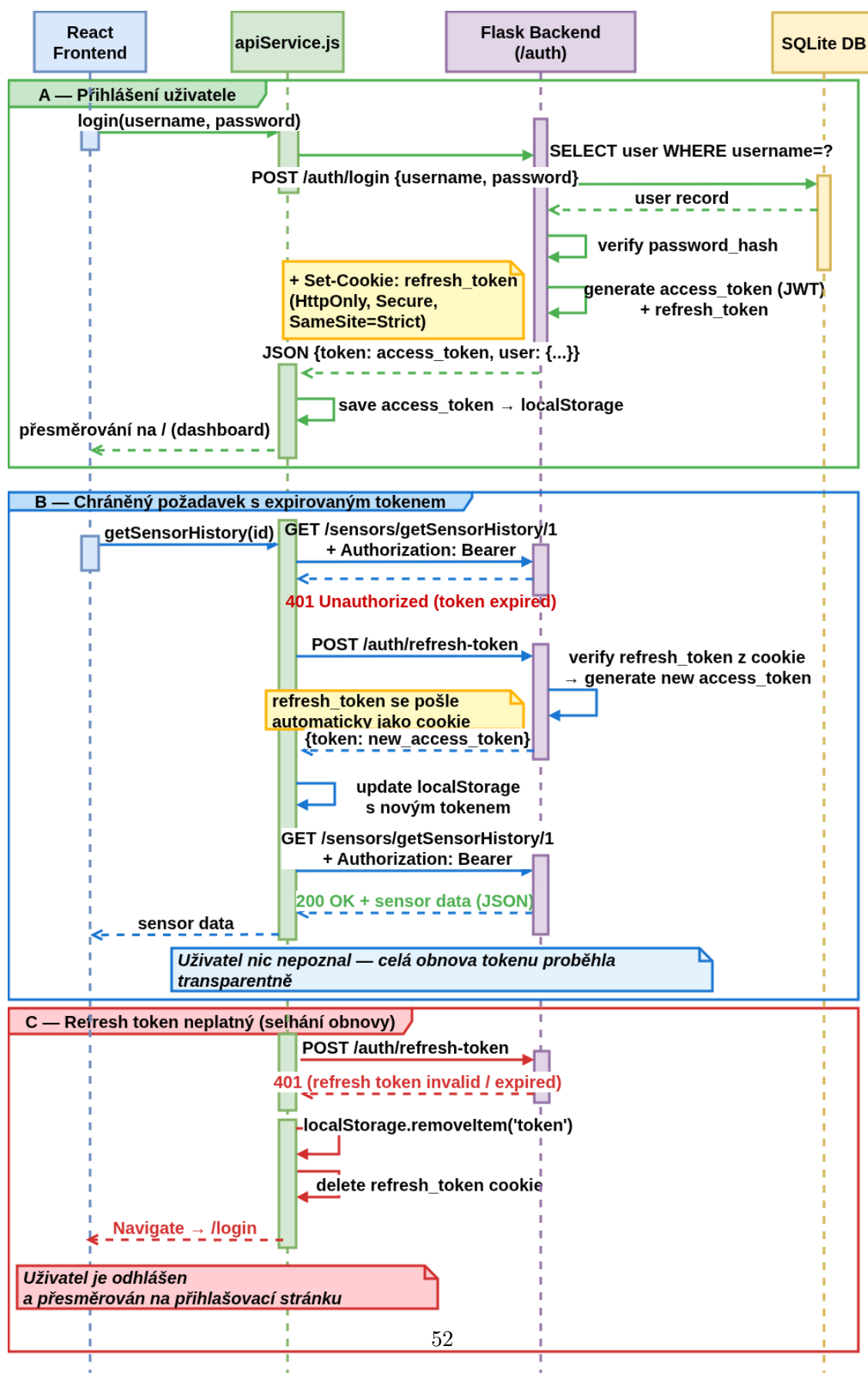
Při přihlášení backend vygeneruje dvojici tokenů. Krátkodobý **access token** (platnost 15 minut) obsahuje identifikátor uživatele, jeho jméno a roli. Je vrácen v těle JSON odpovědi a frontend jej uloží do `localStorage`. Dlouhodobý **refresh token** (platnost 7 dní) je nastaven jako `HttpOnly`, `Secure`, `SameSite=Strict` cookie, takže není přístupný JavaScriptu a je automaticky přikládán k požadavkům prohlížečem.

Pokud access token expiruje, frontend tuto situaci detekuje a automaticky odešle požadavek na obnovu tokenu. Refresh token je přiložen automaticky jako cookie; backend jej ověří a vydá nový access token. Uživatel si celé obnovy nevšimne. Pokud je refresh token neplatný nebo expirovaný, jsou oba tokeny zneplatněny a uživatel je přesměrován na přihlašovací stránku. Tento tok znázorňuje sekvenční diagram na obrázku 6.5.

Autorizace na úrovni dat je řešena kontrolou vlastnictví prostřednictvím vazební tabulky `UserSensor`. Systém rozlišuje dvě role uživatelů – jejich přehled uvádí tabulka 6.1.

Role	Oprávnění	Omezení
user	Správa vlastních senzorů, dashboardů, export dat	Přístup pouze k vlastním senzorům
admin	Vše jako user + správa všech uživatelů	Bez omezení

Tabulka 6.1: Přehled uživatelských rolí a jejich oprávnění



Obrázek 6.5: Sekvenční diagram JWT autentizace a obnovy tokenu

6.2.5 Návrh uživatelského rozhraní

Uživatelské rozhraní bylo navrženo jako jednostránková aplikace (SPA) s pevnou strukturou navigace. Rozložení tvoří boční navigační lišta fixně umístěná na levé straně obrazovky a hlavní obsahová oblast, která vyplňuje zbylý prostor. Navigační lišta je zobrazena na všech chráněných stránkách; na přihlašovací a chybové stránce serveru je záměrně skryta.

Navigace mezi jednotlivými sekcemi aplikace je realizována pomocí knihovny `react-router-dom`. Routy jsou rozděleny na veřejné – přístupné bez přihlášení – a chráněné, které jsou obaleny komponentou `ProtectedRoute`. Tato komponenta implementuje třístavovou logiku ověření přístupu: nejprve ověří dostupnost backendu, poté přítomnost JWT tokenu v `localStorage`. Teprve po úspěšném průchodu oběma kontrolami je zobrazena požadovaná stránka. Přehled navržených rout a jejich přístupových oprávnění shrnuje tabulka 6.2.

Cesta	Komponenta	Přístup	Popis
<code>/login</code>	Login	Veřejná	Přihlášení uživatele
<code>/server-error</code>	ServerError	Veřejná	Chybová stránka serveru
<code>/</code>	Home	Chráněná	Hlavní dashboard
<code>/sensors</code>	SensorsPage	Chráněná	Správa senzorů
<code>/data-export</code>	DataExport	Chráněná	Export naměřených dat
<code>/data-import</code>	DataImport	Chráněná	Import dat do senzoru
<code>/users</code>	AdminPage	Chráněná (admin)	Správa uživatelů
<code>/create-dashboard</code>	DashboardForm	Chráněná	Vytvoření dashboardu
<code>/insertUser</code>	UserForm	Chráněná (admin)	Registrace uživatele

Tabulka 6.2: Přehled navržených rout frontendové aplikace

Navigační lišta podmíněně zobrazuje odkaz na správu uživatelů pouze uživatelům s rolí `admin`, čímž je administrátorská část rozhraní oddělena od běžného uživatelského prostředí. Pro zajištění použitelnosti na mobilních zařízeních byl navržen responzivní mechanismus – na obrazovkách s šířkou menší než 768 px je navigační lišta ve výchozím stavu skryta a zobrazena je pouze ikona hamburgerového menu.

6.3 Implementace

Tato kapitola popisuje konkrétní implementaci navržené architektury. Backendová část aplikace byla realizována ve frameworku Flask v jazyce Python a je organizována do tří hlavních vrstev: routy přijímající HTTP požadavky, servisní třídy zapouzdřující aplikační logiku a databázové modely definované prostřednictvím ORM nástroje SQLAlchemy. Klíčovou komponentou celého systému je třída `ModbusManager`, která zajišťuje komunikaci se senzory přes sběrnici RS-485, správu mezipaměti naměřených hodnot a řízení vláken na pozadí. Kapitola dále popisuje implementaci autentizace pomocí JWT tokenů a přehled REST API endpointů zprostředkovávajících data frontendu.

6.3.1 Nastavení prostředí

Na zařízení Raspberry Pi byl nainstalován operační systém Raspberry Pi OS Lite, tedy varianta bez grafického rozhraní, která šetří systémové prostředky a plně vyhovuje požadavkům projektu. Pro vzdálenou správu zařízení byl využit protokol SSH v rámci lokální sítě.

Backend aplikace byl vyvíjen ve frameworku Flask nainstalovaném do samostatného virtuálního prostředí jazyka Python. Virtuální prostředí izoluje závislosti projektu od systémových balíčků a umožňuje jejich jednoduchou reprodukci příkazem `pip install -r requirements.txt`. Mezi klíčové závislosti patří `Flask`, `SQLAlchemy`, `minimalmodbus`, `PyJWT` a `pytz`.

Frontend byl postaven na knihovně `React.js` ve verzi 18 s využitím build nástroje `Vite`. Závislosti frontendové části jsou spravovány nástrojem `npm` a jejich instalace probíhá příkazem `npm install`.

6.4 Backend

Backend aplikace byla implementována ve frameworku Flask v jazyce Python. Zdrojový kód je organizován do tří hlavních vrstev: adresář `routes/` obsahuje Flask Blueprinty, které přijímají HTTP požadavky a předávají je ke zpracování; adresář `services/` zapouzdřuje aplikační logiku do samostatných tříd; a adresář `models/` definuje databázové entity prostřednictvím ORM nástroje SQLAlchemy. Toto oddělení zodpovědností umožňuje přehlednou správu kódu a nezávislé rozšiřování jednotlivých vrstev.

Pro zajištění modularity byl backend rozdělen do samostatných Flask Blueprintů, z nichž každý obsluhuje konkrétní doménu aplikace. Registrace všech Blueprintů probíhá centrálně ve funkci `init_routes(app)`, která přiřadí každému modulu jeho URL prefix:

Každý z jednotlivých Blueprintů lze vyvíjet a testovat samostatně, protože nemají žádnou závislost na ostatních modulech. Pro přidání nového Blueprintu stačí vytvořit nový a zaregistrovat ho v centrální funkci. Pro každý modul platí jednotný vzor pro zpracovávání požadavků.

Blueprint	URL prefix	Popis
auth_api	/auth	Přihlášení, registrace, správa session
sensors_api	/sensors	CRUD senzorů, historie, export
dashboard_api	/dashboard	Správa dashboardů a widgetů
widget_api	/widget	Operace nad widgety
modbus_api	/modbus	Přímé ovládání Modbus komunikace
user_api	/users	Administrace uživatelů

Tabulka 6.3: Přehled Flask Blueprintů a jejich URL prefixů

6.4.1 ModbusManager

Hlavní komponentou celého backendu je třída **ModbusManager**, která zapouzdřuje veškerou logiku spojenou se sběrem dat ze senzorů. Třída ovládá komunikaci po sběrnici RS-485 prostřednictvím protokolu Modbus RTU, ukládání naměřených hodnot do mezipaměti s následným dávkovým zápisem do databáze a řízení vláken na pozadí, která tyto operace provádějí autonomně. Při inicializaci se z databáze načtou konfigurace všech vytvořených senzorů do paměťové struktury **sensor_map**, pro každý senzor se vytvoří samostatný zámek (**threading.Lock**) a inicializuje se jediný Modbus instrument s konfigurací 8 datových bitů, bez parity a 1 stop bit. Následně se spustí daemon vlákna, která zajišťují automatický sběr a perzistenci dat po celou dobu života aplikace.

6.4.1.1 Komunikace se senzory

Komunikace mezi platformou Raspberry Pi a senzory připojenými prostřednictvím sběrnice RS-485 je realizována knihovnou **minimalmodbus**, která komunikuje přes sériový port **/dev/ttyUSB0**. Přenosová rychlost je nastavována dynamicky před každým čtením podle parametrů konkrétního senzoru, protože různé senzory mohou vyžadovat odlišný baudrate.

Samotné čtení ze senzoru probíhá ve dvou krocích. Nejprve se zamkne zámek konkrétního senzoru a následně sdílený sériový zámek. Poté se nastaví baudrate a zavolá metoda **read_register** s adresou registru, dělicí konstantou a funkčním kódem, jak ukazuje kód. 6.1.

```

1 def read_sensor(self, sensor_id: int):
2     sensor = self.sensor_map.get(sensor_id)
3     if not sensor or not sensor['is_active']:
4         return None
5     with self.sensor_locks[sensor_id]:
6         with self.serial_lock:
7             self.instrument.serial.baudrate = sensor['baudrate']
8             time.sleep(0.05)
9             value = self.instrument.read_register(

```

```
10         sensor['address'], sensor['scaling'],
11         sensor['functioncode'])
12     return float(value)
```

Listing 6.1: Čtení hodnoty ze senzoru přes Modbus

V případě selhání komunikace (timeout 1 sekunda, maximálně 3 opakované pokusy) se systém automaticky pokusí načíst poslední známou hodnotu z databáze. Tento dvoustupňový fallback mechanismus zajišťuje, že uživatel vždy obdrží data — buď aktuální z Modbusu, nebo nejnovější historická z databáze.

6.4.1.2 Ukládání dat ze senzorů a mezipaměť

Návrh ukládání naměřených dat vychází z omezení cílové platformy. Raspberry Pi využívá jako primární úložiště microSD kartu, která má omezenou životnost vyjádřenou počtem zápisových cyklů. Každý zápis do databáze SQLite generuje I/O operace na úrovni souborového systému — samotný zápis dat, aktualizaci žurnálu (WAL nebo rollback journal) a synchronizaci na disk. Při přímém ukládání každého jednotlivého měření by systém se čtyřmi senzory a vzorkovací frekvencí 10 sekund generoval přibližně 34 500 zápisových operací denně, což by výrazně zkracovalo životnost karty.

Z tohoto důvodu byla implementována dvouúrovňová mezipaměť. Na první úrovni se nachází paměťová fronta (`queue.Queue`), do které vlákno čtení ukládá naměřené hodnoty okamžitě po jejich získání ze senzoru. Na druhé úrovni je ring buffer (`collections.deque` s parametrem `maxlen`), který slouží jako cyklický zásobník s pevně omezenou kapacitou. Při zaplnění se nejstarší záznamy automaticky přepisují novými, čímž je zajištěno konstantní využití paměti bez rizika jejího nekontrolovaného růstu.

```
1 def _add_to_cache(self, sensor_id: int, value: float):
2     data_point = {
3         'sensor_id': sensor_id,
4         'value': value,
5         'timestamp': datetime.now(local_tz)
6     }
7     try:
8         self.data_queue.put_nowait(data_point)
9     except queue.Full:
10         self._force_flush_batch()
11         try:
12             self.data_queue.put_nowait(data_point)
13         except queue.Full:
14             if self.memory_cache:
15                 self.memory_cache.popleft()
16                 self.memory_cache.append(data_point)
```

Pokud je fronta plná, systém nejprve vynutí okamžitý zápis části dat do databáze metodou `_force_flush_batch`, čímž uvolní místo. Pokud ani po tomto zásahu nelze data zařadit, jsou vložena přímo do ring bufferu s odstraněním nejstaršího záznamu. Tento kaskádový mechanismus zajišťuje, že žádné měření není zahozeno bez pokusu o uložení.

Samotný zápis do databáze probíhá dávkově. Metoda `_flush_batch` převezme seznam naměřených hodnot, vytvoří z nich objekty `SensorData` a uloží je jediným voláním `db.session.bulk_save_objects` následovaným `db.session.commit`. Díky dávkovému přístupu se místo desítek individuálních zápisů provede jedna transakce, což snižuje počet I/O operací na SD kartě snižuje. V případě chyby při zápisu je transakce vrácena zpět voláním `db.session.rollback`, aby nedošlo k nekonzistenci dat.

Kromě periodického zápisu poskytuje `ModbusManager` metodu `get_latest_data`, která umožňuje API endpointům získat poslední naměřenou hodnotu bez nutnosti komunikace po sběrnici. Metoda prohledává ring buffer od nejnovějších záznamů směrem k nejstarším a vrací první hodnotu pro daný senzor, jejíž časové razítko nepřesahuje konfigurovatelný práh stáří (výchozí 5 minut). Pokud cache neobsahuje dostatečně aktuální záznam, je automaticky spuštěno nové čtení ze senzoru. Tento mechanismus snižuje zatížení sběrnice při opakovaných dotazech z frontendu a zároveň garantuje, že vrácená data nejsou zastaralá.

6.4.1.3 Plánované úlohy a vlákna na pozadí

Systém využívá tři daemon vlákna spuštěná při inicializaci `ModbusManageru`, která běží na pozadí po celou dobu života aplikace. První vlákno (`_auto_read_loop`) zajišťuje periodické čtení ze všech aktivních senzorů podle jejich individuálních intervalů vzorkování. Samotné čtení neprobíhá sekvenčně v rámci tohoto vlákna, ale je delegováno do `ThreadPoolExecutor` s kapacitou čtyř pracovních vláken prostřednictvím volání `executor.submit`. Toto řešení umožňuje paralelní obsluhu více senzorů, přičemž sdílený sériový port je chráněn zámkem `serial_lock`, který zajišťuje serializaci fyzického přístupu.

```
1 def _auto_read_loop(self):
2     while not self._shutdown_event.is_set():
3         now = time.time()
4         for sensor_id, sensor in self.sensor_map.items():
5             if not sensor["is_active"]:
6                 continue
7             if now - self.last_read.get(sensor_id, 0.0) \
8                 >= sensor["sampling_rate"]:
9                 self.executor.submit(self.read_sensor, sensor_id)
10        time.sleep(self.check_interval)
```

Druhé vlákno (`_periodic_flush_background`) v konfigurovatelném intervalu kontroluje obsah ring bufferu a při dosažení nastaveného prahu (`batch_size`) zahajuje dávkový zápis do databáze.

Oddělení čtení od zápisu zabraňuje tomu, aby latence databázových operací ovlivňovala frekvenci sběru dat ze senzorů. Třetí vlákno (`_stats_monitor_background`) monitoruje statistiky úspěšnosti čtení a loguje je pro diagnostické účely.

Všechna vlákna jsou spuštěna jako daemon, což znamená, že se automaticky ukončí při vypnutí hlavního procesu. Pro řízené ukončení slouží sdílený objekt `threading.Event` (`_shutdown_event`), jehož nastavení signalizuje všem vláknům požadavek na ukončení. Metoda `shutdown` poté počká na dokončení všech rozpracovaných úloh v thread poolu, provede finální zápis zbývajících dat z ring bufferu do databáze a uzavře sériový port. Tato sekvence je důležitá zejména pro prostředí Raspberry Pi, kde neočekávané přerušení bez uvolnění portu může zablokovat následující inicializaci Modbus komunikace.

6.4.2 Uživatelé a autentizace

Systém implementuje dvouúrovňový mechanismus řízení přístupu založený na JWT tokenech a uživatelských rolích. Při přihlášení odešle uživatel přihlašovací údaje na endpoint `/auth/login`. Backend ověří heslo pomocí `werkzeug.security.check_password_hash`, vygeneruje dvojici tokenů — krátkodobý access token (JWT obsahující identifikátor uživatele, jméno a roli) a dlouhodobý refresh token. Access token je vrácen v těle JSON odpovědi a frontend jej uloží do `localStorage`. Refresh token je nastaven jako `HttpOnly`, `Secure`, `SameSite=Strict` cookie, takže není přístupný JavaScriptu a je automaticky přikládán k požadavkům prohlížečem.

Ověření přihlášení zajišťuje dekorátor `@login_required`, aplikovaný na všechny chráněné endpointy. Dekorátor extrahuje access token z hlavičky `Authorization: Bearer <token>`, ověří jeho platnost a naplní session údaji o uživateli. Neplatný nebo chybějící token vrátí odpověď s HTTP kódem 401.

```
1 def login_required(f):
2     @wraps(f)
3     def decorated(*args, **kwargs):
4         token = request.headers.get('Authorization')
5         if not token:
6             return jsonify({'error': 'Token missing'}), 401
7         token_data = AuthService.verify_token(
8             token.split(' ')[1])
9         if not token_data:
10            return jsonify({'error': 'Invalid token'}), 401
11         session['user_id'] = token_data['user_id']
12         return f(*args, **kwargs)
13     return decorated
```

Pokud access token expiruje, vrátí backend kód 401. Frontend (`apiService.js`) tuto situaci detekuje a automaticky odešle požadavek na `POST /auth/refresh-token`. Refresh token je při-

ložen automaticky jako cookie; backend jej ověří a vydá nový access token. Frontend aktualizuje `localStorage` a zopakuje původní požadavek — uživatel si celé obnovy nevšimne. Pokud je refresh token neplatný nebo expirovaný, jsou oba tokeny zneplatněny a uživatel je přesměrován na přihlašovací stránku.

Autorizace na úrovni dat je řešena kontrolou vlastnictví prostřednictvím vazební tabulky `UserSensor`. Metoda `_ensure_user_owns_sensor()` ve třídě `SensorService` ověří, zda senzor patří přihlášenému uživateli, a v případě nedostatečného oprávnění vyvolá výjimku `PermissionError`. Administrátorská role (`admin`) má rozšířená oprávnění pro správu uživatelů, přístup k nim kontroluje funkce `_admin_only()`.

6.4.3 REST API endpointy

Zprostředkování dat uživatelům bylo realizováno prostřednictvím REST API ve formátu JSON. Frontend komunikuje s backendem prostřednictvím definovaných endpointů, které pokrývají všechny případy užití systému. Kompletní přehled endpointů je uveden v příloze jako tabulka A.1a. Každý endpoint dodržuje jednotný vzor zpracování: po přijetí požadavku se ověří autentizace, extrahují se parametry a zpracování se předá příslušné třídě ve vrstvě `services/`. Odpověď je vždy ve formátu JSON a obsahuje buď požadovaná data s kódem 200, nebo chybovou zprávu s odpovídajícím HTTP kódem (400, 403, 404, 500).

6.5 Frontend

Frontendová část aplikace byla implementována v knihovně React.js verze 18 s využitím build nástroje Vite. Aplikace byla navržena jako jednostránková (SPA — Single Page Application), což znamená, že po prvním načtení stránky probíhá veškerá navigace na straně klienta bez opětovného načítání celé stránky ze serveru. Komponentový přístup Reactu umožnil rozdělit uživatelské rozhraní do znovupoužitelných celků, kde každá komponenta zapouzdřuje svůj stav, logiku i vizuální podobu.

Zdrojový kód frontendu je organizován do následující adresářové struktury:

```
src/  
  components/ # znovupouzitelne UI komponenty  
  features/   # funkcní celky (auth, dashboard)  
  pages/      # stránkové komponenty (Home, SensorsPage, ...)  
  api/        # API komunikace (apiService.js)  
  hooks/      # vlastní React hooky (useApi)  
  utils/      # pomocné funkce (chartUtils.js)  
  App.jsx     # kořenová komponenta se smerováním
```

6.5.1 Směrování a ochrana rout

Navigace mezi jednotlivými sekcemi aplikace byla realizována pomocí knihovny **react-router-dom** ve verzi 6. Celá aplikace je obalena komponentou **BrowserRouter**, která zajišťuje synchronizaci URL adresy v prohlížeči s aktuálně zobrazenou stránkou. Definice všech dostupných rout se nachází v kořenové komponentě **App.jsx**, kde každá cesta odpovídá konkrétní stránkové komponentě. Jejich přehled a přístupové oprávnění shrnuje tabulka 6.2.

Většina rout je chráněna komponentou **ProtectedRoute**, která implementuje třístavovou logiku ověření přístupu. Při každém pokusu o zobrazení chráněné stránky komponenta nejprve ověří dostupnost backendu voláním endpointu **/auth/status**. Pokud server neodpovídá, uživatel je přesměrován na stránku **/server-error**. V případě dostupného serveru se následně kontroluje přítomnost JWT tokenu v **localStorage**. Nepřihlášený uživatel je přesměrován na přihlašovací stránku. Teprve po úspěšném průchodu oběma kontrolami je zobrazena požadovaná stránka.

```
1 const ProtectedRoute = ({ children }) => {  
2   const isAuthenticated = localStorage.getItem("token") !== null;  
3   const [isServerAvailable, setIsServerAvailable] = useState(null);  
4  
5   useEffect(() => {  
6     const checkServer = async () => {  
7       try {  
8         await callApi(() => api.get("/auth/status"));
```

```

9      setIsServerAvailable(true);
10    } catch (error) {
11      setIsServerAvailable(false);
12    }
13  };
14  checkServer();
15 }, []);
16
17 if (isServerAvailable === null) return <div>Loading...</div>;
18 if (!isServerAvailable) return <Navigate to="/server-error" />;
19 if (!isAuthenticated) return <Navigate to="/login" />;
20 return children;
21 };

```

Listing 6.2: Komponenta ProtectedRoute

Navigační lišta (NavBar) je zobrazena na všech chráněných stránkách, avšak na přihlašovací a chybové stránce serveru je záměrně skryta. Toto chování je řízeno v komponentě `AppContent`, která pomocí hooku `useLocation` zjistí aktuální cestu a na základě pole vyloučených cest rozhodne, zda komponentu `NavBar` vykreslit. Navigační lišta dále podmíněně zobrazuje odkaz na správu uživatelů pouze uživatelům s rolí `admin`, čímž je zajištěno oddělení administrátorských funkcí od běžného uživatelského rozhraní.

6.5.2 Komunikace s backendem

Veškerá komunikace mezi frontendovou aplikací a Flask backendem probíhá prostřednictvím REST API požadavků ve formátu JSON. Pro centralizaci této logiky byl vytvořen modul `apiService.js`, který zapouzdřuje všechna HTTP volání do jediného objektu `api`. Jádrem modulu je funkce `apiRequest`, která zajišťuje jednotné zpracování požadavků: automaticky připojuje JWT token z `localStorage` do hlavičky `Authorization`, nastavuje hlavičky `Content-Type` a `Accept`, a zpracovává chybové stavy.

Důležitou součástí komunikační vrstvy je mechanismus automatické obnovy tokenu. Pokud server vrátí stavový kód 401 (neautorizovaný přístup), funkce `apiRequest` se pokusí získat nový access token voláním endpointu `/auth/refresh-token`. Refresh token je uložen v HTTP-only cookie, což zamezuje jeho čtení z JavaScriptu a zvyšuje bezpečnost. V případě úspěšné obnovy je původní požadavek automaticky zopakován s novým tokenem. Pokud obnova selže, uživatel je odhlášen a přesměrován na přihlašovací stránku.

Pro ošetření výpadků serveru v rámci komponent chráněných rout byl vytvořen vlastní hook `useApi`. Tento hook obaluje libovolné API volání a v případě, že server není dostupný (chyba typu `NetworkError`), automaticky přesměruje uživatele na stránku `/server-error`. Ostatní chyby jsou předávány volající komponentě ke zpracování.

Přehled klíčových API volání, která modul `apiService.js` zpřístupňuje, shrnuje tabulka 6.4.

Funkce	Metoda a endpoint	Účel
<code>login</code>	POST <code>/auth/login</code>	přihlášení uživatele
<code>register</code>	POST <code>/auth/register</code>	registrace uživatele
<code>getDashboards</code>	GET <code>/dashboard/userDashBoards</code>	načtení dashboardů
<code>getDashboardWidgets</code>	GET <code>/dashboard/widgets/{id}</code>	widgety dashboardu
<code>createWidget</code>	POST <code>/dashboard/widget</code>	vytvoření widgetu
<code>getSensorHistory</code>	GET <code>/sensors/getSensorHistory/{id}</code>	historie senzoru
<code>getLatestSensorData</code>	GET <code>/sensors/getLatestSensorData/{id}</code>	aktuální hodnota
<code>exportSensorData</code>	POST <code>/sensors/export_data</code>	export dat
<code>toggleSensorActive</code>	PATCH <code>/sensors/{id}/toggle-active</code>	aktivace senzoru
<code>saveWidgetPositions</code>	POST <code>/dashboard/.../save_positions</code>	uložení pozic

Tabulka 6.4: Přehled klíčových API volání modulu `apiService`

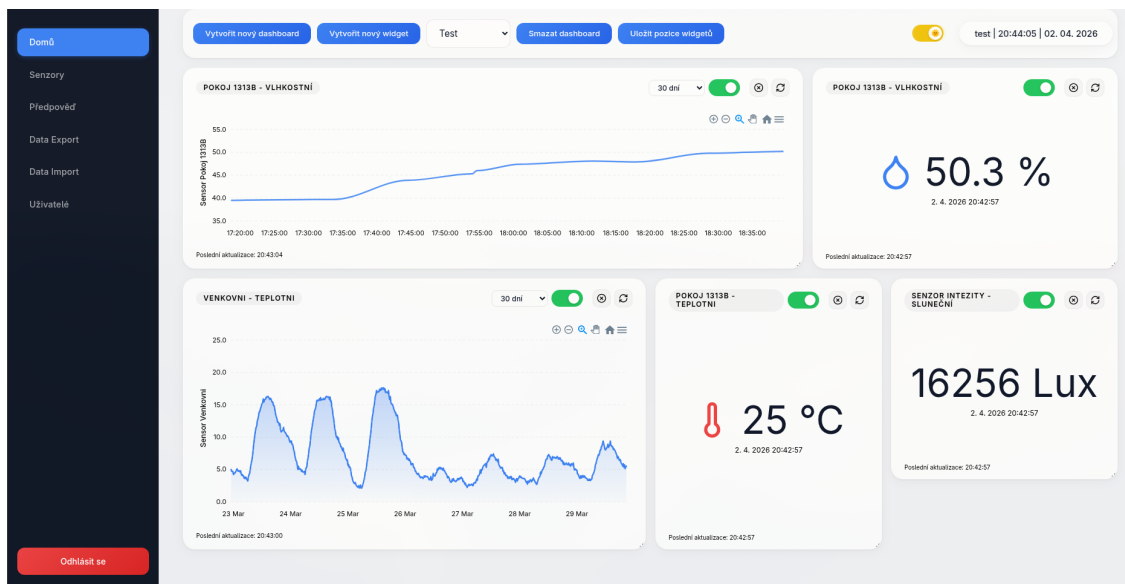
Pro ošetření výpadků serveru v rámci komponent chráněných rout byl vytvořen vlastní hook `useApi`. Tento hook obaluje libovolné API volání a v případě, že server není dostupný (chyba typu `NetworkError`), automaticky přesměruje uživatele na stránku `/server-error`. Ostatní chyby jsou předávány volající komponentě ke zpracování.

6.5.3 Interaktivní dashboard a widgety

Hlavní funkci aplikace představuje interaktivní dashboard, který uživateli umožňuje sledovat data ze senzorů prostřednictvím konfigurovatelných widgetů. Implementace dashboardu je soustředěna v komponentě `Dashboard.jsx` a navazujících komponentách pro widgety a formuláře.

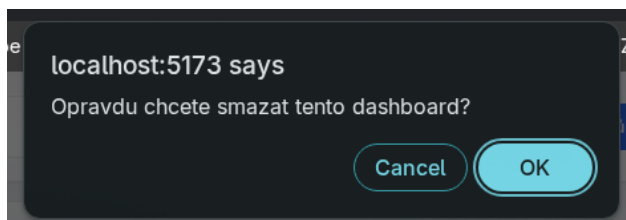
Po přihlášení aplikace načte seznam dashboardů přiřazených aktuálnímu uživateli. Každý uživatel může vlastnit více dashboardů a přepínat mezi nimi. Vytvoření nového dashboardu probíhá prostřednictvím formuláře `DashboardForm`, kde uživatel zadá název a volitelný popis.

Pro rozložení widgetů na ploše dashboardu byla využita knihovna `react-grid-layout`. Tato knihovna poskytuje responzivní mřížkový systém, ve kterém může uživatel jednotlivé widgety přesouvat pomocí drag-and-drop a měnit jejich velikost tažením za okraj. Pozice a rozměry widgetů jsou po změně automaticky odesílány na backend prostřednictvím endpointu `save_positions`, čímž je zajištěna jejich perzistence mezi jednotlivými sezeními. Dashboard vytvořený pro ilustraci můžeme vidět na obrázku 6.6.



Obrázek 6.6: Hlavní Dashboard

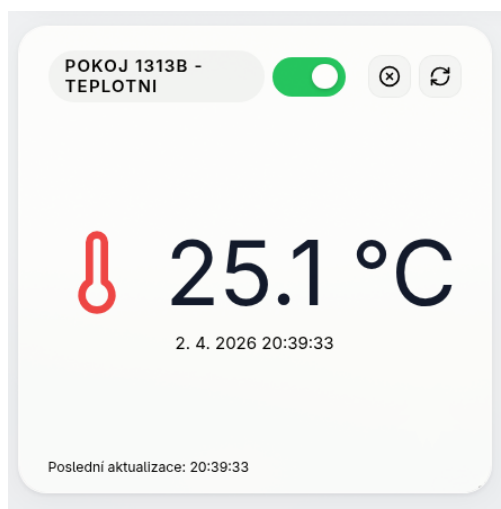
Odstranění dashboardu je podmíněno potvrzením uživatele prostřednictvím standardního dialogu prohlížeče (funkce `window.confirm`), jak ukazuje obrázek 6.7. Tato kontrola zajišťuje, že nedojde k nechtěnému smazání dat v důsledku neúmyslného kliknutí. Po potvrzení záměru je volán endpoint `deleteDashboard` s identifikátorem aktuálního dashboardu, načež je seznam dashboardů znovu načten a vybrán první dostupný dashboard.



Obrázek 6.7: Potvrzovací dialog pro smazání dashboardu

Přidání nového widgetu probíhá přes formulář `WidgetForm`. V rámci formuláře si uživatel vybere připojený senzor ze seznamu a zvolí typ vizualizace. Aplikace podporuje čtyři typy widgetů: *Line Chart* (čárový graf), *Area Chart* (plošný graf), *Gauge* a *Value* (zobrazení aktuální hodnoty). Teplotní a vlhkostní senzory jsou reprezentovány ikonou z knihovny `lucide-react`.

Samotná komponenta `Widget` je zodpovědná za načtení dat ze senzoru, jejich zpracování a vykreslení grafu. Pro grafové typy (`line` a `area`) komponenta volá endpoint `getSensorHistory` s parametrem časového rozsahu, který uživatel volí pomocí rozbalovacího menu (24 hodin, 7 dní, 30 dní). Pro typ `value` (obrázek 6.8) se volá endpoint `getLatestSensorData`, který vrací pouze poslední naměřenou hodnotu.



Obrázek 6.8: Teplotní Widget typy value

Aby bylo vykreslování grafů plynulé i při velkém objemu dat, implementuje komponenta vzorkování pomocí hooku `useMemo`. Na základě zvoleného časového rozsahu je stanoven maximální počet zobrazených bodů (například 144 bodů pro 24 hodin). Pokud skutečný počet záznamů přesahuje tento limit, data jsou rovnoměrně navzorkována. Pro vykreslování grafů byla použita knihovna *Apex-Charts* prostřednictvím její React obálky `react-apexcharts`. Konfigurace grafů je oddělena do modulu `chartUtils.js`, kde jsou definovány funkce `getLineChartOptions` a `getAreaChartOptions`. Obě funkce sdílejí společnou základní konfiguraci (`getBaseChartOptions`) a rozšiřují ji o specifické vlastnosti daného typu grafu, jako je typ výplně u plošného grafu nebo formát os.

Každý widget dále poskytuje ovládací prvky: přepínač pro aktivaci a deaktivaci senzoru, tlačítko pro manuální obnovení dat a tlačítko pro odstranění widgetu z dashboardu. Stav aktivace senzoru je synchronizován s backendem voláním endpointu `toggleSensorActive`.

6.5.4 Správa senzorů

Stránka `SensorsPage` poskytuje uživateli přehled všech senzorů, které jsou mu přiřazeny. Senzory jsou zobrazeny v tabulce se sloupci pro název, typ, měřicí jednotku a stav aktivity. Stav aktivity je ovládán přepínačem (toggle switch), který okamžitě aktualizuje stav senzoru na backendu. Uživatel může rovněž upravit parametry existujícího senzoru prostřednictvím formuláře `SensorForm` (obrázek 6.9) nebo přidat nový senzor pomocí modálního okna `AddSensorModal`.

Modální okno nabízí dva režimy: přiřazení již existujícího senzoru ze systému nebo vytvoření zcela nového. Při vytváření nového senzoru je k dispozici volitelný výběr šablony z existujících senzorů – po jejím zvolení jsou pole formuláře předvyplněna hodnotami vybraného senzoru (typ, adresa, funkční kód, baudrate, scaling), které uživatel může dále upravit. Tím je naplněna podmínka zadání, umožňující odvození nového senzoru z již existujícího.

Upravit senzor

NÁZEV

Pokoj 1313B

TYP SENZORU

Teplotni

ADRESA

1

FUNCTION CODE

4

BIT

9600

SCALING

1

JEDNOTKA

°C

Obrázek 6.9: Formulář na úpravu senzoru

Pokud chce uživatel odebrat jeden ze svých senzorů, může tak učinit pomocí sloupce akcí, kde se nachází tlačítko pro smazání. Po potvrzení akce je odstraněn pouze záznam z tabulky **UserSensor** – senzor jako takový zůstává v systému a nadále je dostupný ostatním uživatelům, kteří jej mají přiřazený.

Trvalé odstranění senzoru ze systému je vyhrazeno výhradně administrátorovi a provádí se na stránce **AdminPage**. Před samotným smazáním systém ověří, zda senzor neobsahuje naměřená data – pokud ano, operace je zamítnuta a uživatel je vyzván, aby data nejprve smazal nebo exportoval prostřednictvím funkce pro export dat. Teprve po splnění této podmínky může administrátor senzor smazat. Při mazání jsou kaskádově odstraněny všechny navázané záznamy: nejprve jsou nalezeny všechny widgety přiřazené danému senzoru, následně jsou smazány odpovídající záznamy z tabulek **DashboardWidget** a **Widget**. Poté jsou odstraněny všechny záznamy z tabulky **UserSensor** a nakonec je smazán samotný záznam ze tabulky **Sensor**.

6.5.5 Předpověď počasí

Stránka `WeatherPage` rozšiřuje aplikaci o zobrazení meteorologických dat z externího zdroje a jejich porovnání s hodnotami naměřenými vlastními senzory systému. Veškerá komunikace s *Open-Meteo API* probíhá přes Flask backend – endpoint `/weather` obsluhovaný modulem `weather_route.py`. Frontend volá funkci `api.getWeatherForecast` z modulu `apiService.js`, která předává parametry lokality a časového rozsahu. Backend v závislosti na zvoleném rozsahu automaticky volí mezi předpovědním a archivním rozhraním Open-Meteo a vrací denní hodnoty (maximální a minimální teplota, východ a západ slunce, délka dne) a hodinové hodnoty (teplota a relativní vlhkost vzduchu).

Na straně backendu zajišťuje zpracování požadavku třída `WeatherService`. Ta v závislosti na zvoleném časovém rozsahu volí mezi dvěma URL adresami *Open-Meteo API*: pro aktuální a budoucí data je použita adresa předpovědi (`forecast`), zatímco pro historická data (end datum leží před dneškem) je použita adresa archivního rozhraní (`archive-api.open-meteo.com`). Aby bylo omezeno množství síťových požadavků, využívá `WeatherService` knihovnu `requests-cache` s dobou platnosti cache jednu hodinu. Backend vrací strukturovaná data obsahující denní hodnoty (maximální a minimální teplota, časy východu a západu slunce, délka dne) a hodinové hodnoty (teplota a relativní vlhkost vzduchu).

Před zobrazením dat musí uživatel zadat lokalitu. Komponenta při svém načtení nejprve zavolá endpoint `/users/me/location` a pokusí se načíst uloženou výchozí polohu uživatele z backendu. Pokud je uložená poloha nalezena, je automaticky předvyplněna do vstupního pole a provedeno okamžité načtení dat. Uživatel může výchozí polohu kdykoliv změnit ručním vyhledáváním – při zadávání textu do vyhledávacího pole volána funkce `api.searchWeatherLocations`, která dotazuje *Open-Meteo Geocoding API* přímo z frontendu a zobrazuje návrhy lokalit. Po výběru konkrétní lokality může uživatel tuto polohu uložit jako novou výchozí prostřednictvím tlačítka „Uložit jako výchozí polohu“. Uživatel dále volí časový rozsah pomocí dvojice datových polí *Od* a *Do*, jak je patrné z obrázku 6.10.

The screenshot shows a web form titled "Předpověď počasí". It contains a search bar for "Lokalita" with the text "Ostrava-jí" entered. Below the search bar, a dropdown menu displays "Ostrava-Jih, Moravskoslezský kraj, Czechia" with a small "x" icon to its right. To the right of the search bar is a blue button labeled "Načíst počasí". Below the location field are two date pickers labeled "Od" and "Do", both showing the date "04/22/2026".

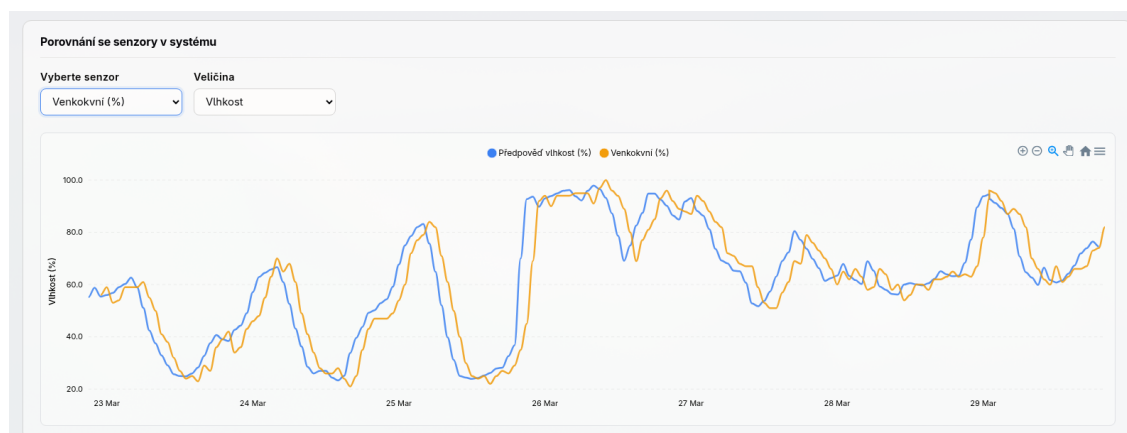
Obrázek 6.10: Vyhledávání lokality s našeptávačem a výběr časového rozsahu

Po úspěšném načtení dat jsou zobrazeny dvě sekce výsledků. První sekce představuje denní přehled ve formě mřížky karet, kde každá karta odpovídá jednomu dni vybraného rozsahu a zobrazuje datum, maximální a minimální teplotu, časy východu a západu slunce a délku dne v hodinách a minutách. Druhá sekce zobrazuje hodinový průběh teploty. Oba výstupy jsou patrné na obrázku 6.11.



Obrázek 6.11: Předpověď počasí pro konkrétní období v zadané lokalitě

Stránka dále nabízí funkci porovnání meteorologických dat s hodnotami senzorů přiřazenými přihlášenému uživateli. Uživatel vybere konkrétní senzor z rozbalovacího seznamu a zvolí veličinu ke srovnání – teplotu nebo relativní vlhkost. Komponenta poté zavolá endpoint `getSensorHistoryHourly` s odpovídajícím časovým rozsahem a výsledky vykreslí do kombinovaného grafu společně s hodinovými daty z Open-Meteo API. Tím umožňuje uživateli vizuálně posoudit shodu mezi venkovní předpovědí a skutečně naměřenými hodnotami ze senzorů, jak ukazuje obrázek 6.12.



Obrázek 6.12: Porovnání hodinové teploty z Open-Meteo s hodnotami senzoru systému a denních karet

6.5.6 Export a import dat

Pro export naměřených dat slouží stránka **DataExport**. Uživatel zde zvolí časový rozsah (datum od, datum do), vybere jeden nebo více senzorů pomocí zaškrťovacích polí, zadá volitelný název souboru a zvolí formát exportu – JSON nebo CSV. Po odeslání požadavku backend vrátí data, ze kterých frontend vytvoří objekt **Blob** a pomocí dočasného odkazu (`URL.createObjectURL`) iniciuje stažení souboru do zařízení uživatele. Tento přístup nevyžaduje přesměrování na novou stránku a stažení proběhne na pozadí aktuální relace.

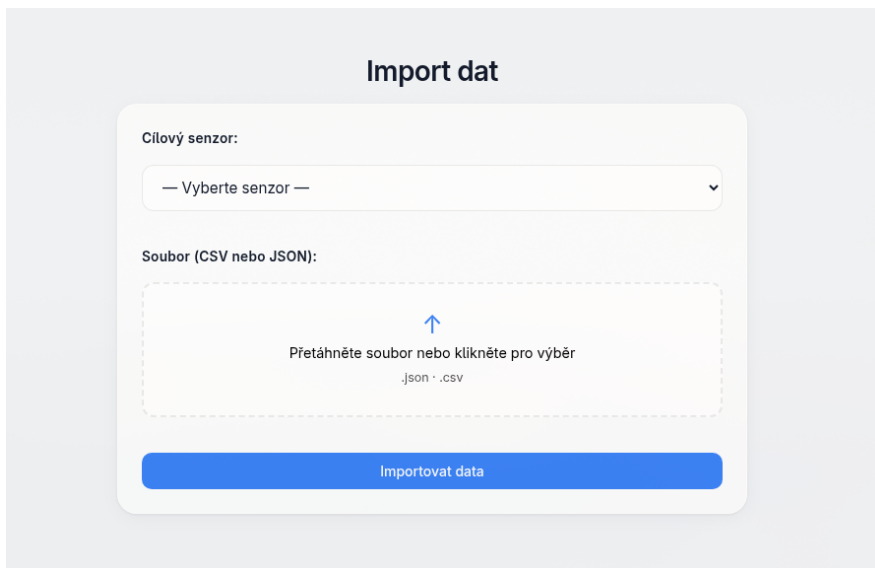
The screenshot shows a web form titled "Export dat". It contains the following elements:

- Datum od:** A date picker with the placeholder text "mm/dd/yyyy, --:-- --".
- Datum do:** A date picker with the placeholder text "mm/dd/yyyy, --:-- --".
- Vyberte senzory:** A list of sensors with checkboxes:
 - ☐ Venkovní (°C)
 - ☐ Venkovní (%)
 - ☐ Senzor Intezity (Lux)
 - ☐ Pokoj 1313B (°C)
- Název souboru (nepovinné):** A text input field with the placeholder text "Pokud nevyplníte, použije se výchozí název".
- Formát exportu:** A dropdown menu currently showing "JSON".
- Exportovat data:** A prominent blue button at the bottom.

Obrázek 6.13: Export dat z senzorů

Uživatel může také importovat data. Pro to slouží stránka **DataImport**. Uživatel nejprve vybere cílový senzor, do kterého mají být data importována, a poté nahraje soubor ve formátu JSON nebo CSV – buď kliknutím, nebo přetažením do vyhrazené zóny. Soubor je zpracován přímo na straně klienta: JSON musí obsahovat pole objektů s atributy `timestamp` a `value`, CSV musí mít hlavičkový řádek s těmito sloupci (oddělovač může být tabulátor nebo čárka). Po úspěšném parsování

je zobrazen náhled prvních pěti záznamů spolu s celkovým počtem. Po potvrzení jsou záznamy ve formátu `timestamp`, `value` odeslány na backend, kde jsou validována přístupová práva uživatele k danému senzoru a záznamy jsou zapsány do databáze. Výsledek operace je zobrazen uživateli jako potvrzovací zpráva s počtem importovaných záznamů.



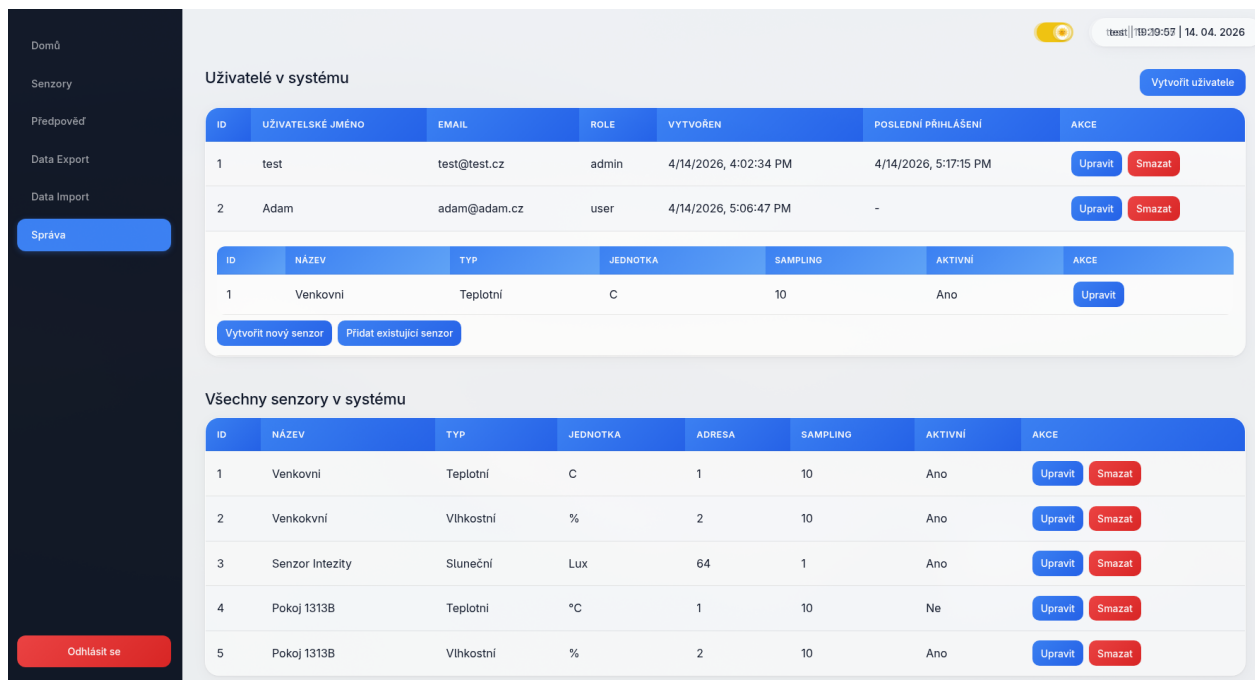
Obrázek 6.14: Import dat do vybraného senzoru

6.5.7 Administrátorská správa

Administrátorská stránka `AdminPage` je přístupná pouze uživatelům s rolí `admin`. Odkaz na tuto stránku je v navigační liště podmíněně zobrazen na základě role aktuálně přihlášeného uživatele, která je uložena v `localStorage`.

Stránka zobrazuje tabulku všech registrovaných uživatelů se sloupci pro identifikátor, uživatelské jméno, e-mail, roli, datum vytvoření účtu a poslední přihlášení. Administrátor může uživatele upravovat přímo v tabulce (inline editace), měnit jejich roli mezi hodnotami `user` a `admin`, nebo uživatele smazat po potvrzení dialogem. Vytvoření nového uživatele probíhá prostřednictvím formuláře `UserForm` na samostatné stránce.

Kliknutím na řádek uživatele v tabulce se rozbalí vnořená tabulka s přehledem senzorů přiřazených danému uživateli (obrázek 6.15). Tato vnořená tabulka zobrazuje identifikátor, název, typ, jednotku, vzorkovací frekvenci a stav aktivity každého senzoru. Administrátor může parametry těchto senzorů rovněž editovat přímo v tabulce. Data senzorů jsou načítána asynchronně při prvním rozbalení řádku voláním endpointu `/users/{id}/sensors` a následně uložena do lokálního stavu, aby se při opakovaném rozbalení nemusela znovu načítat.



Obrázek 6.15: Stránka po správu uživatelů

6.5.8 Rozložení a responzivní design

Rozložení aplikace tvoří boční navigační lišta (NavBar) o šířce 250 px, fixně umístěná na levé straně obrazovky, a hlavní obsahová oblast, která vyplňuje zbylý prostor. Navigační lišta obsahuje odkazy na jednotlivé sekce aplikace a tlačítko pro odhlášení umístěné u spodního okraje.

Pro zajištění použitelnosti na mobilních zařízeních byl implementován responzivní mechanismus pomocí CSS media query. Na obrazovkách s šířkou menší než 768 px je navigační lišta ve výchozím stavu skryta pomocí transformace `translateX(-100%)` a zobrazena je pouze ikona hamburgerového menu. Po kliknutí na tuto ikonu se navigace vysune animovaným přechodem. Tento stav je řízen React stavem `isOpen` v komponentě NavBar.

Vizuální konzistence aplikace je zajištěna pomocí CSS custom properties (proměnných), které definují barvy pro primární prvky, pozadí, text a povrchy. Aplikace podporuje světlý i tmavý režim přepínáním třídy `dark-mode` na kořenovém elementu, přičemž hodnoty proměnných se odpovídajícím způsobem přepíše.

6.6 Ověření funkčnosti navržené aplikace

Součástí zadání bakalářské práce je ověření funkčnosti navržené aplikace. Tato kapitola popisuje sled kroků, které byly provedeny za účelem verifikace správného chování systému v různých provozních podmínkách. Ověření je rozděleno do dvou oblastí: analýza chování systému při výpadku komunikace se senzory a nedostupnosti serveru; a porovnání naměřených dat s referenčními hodnotami z veřejně dostupného meteorologického API. Cílem je prokázat, že aplikace je provozuschopná, odolná vůči chybovým stavům a poskytuje věrohodná data.

6.6.1 Výpadek komunikace

Při provozu IoT systému může dojít k různým druhům výpadků: senzor přestane odpovídat na sběrnici RS-485, dojde k výpadku samotného serveru, nebo se přeruší síťové spojení mezi frontendovou aplikací a backendem. Systém musí na každý z těchto stavů reagovat předvídatelným způsobem a informovat uživatele o vzniklé situaci. Tato podkapitola popisuje sled zpracování chybových stavů na úrovni backendu i frontendu.

Chybové odpovědi backendu

Backend dodržuje jednotný formát chybových odpovědí ve formátu JSON. Každá odpověď obsahuje klíč `status` s hodnotou `"error"` a klíč `message` s popisem chyby. HTTP stavové kódy jsou použity v souladu se standardem REST:

- **400** – neplatný vstup (chybějící povinná pole, nesprávný formát dat),
- **401** – neautorizovaný přístup (neplatný nebo expirovaný JWT token),
- **403** – nedostatečné oprávnění (přístup k administrátorským funkcím bez role `admin`),
- **404** – zdroj nenalezen (neexistující senzor, uživatel, dashboard),
- **500** – interní chyba serveru (neočekávaná výjimka při zpracování).

6.6.1.1 Výpadek senzoru na sběrnici RS-485

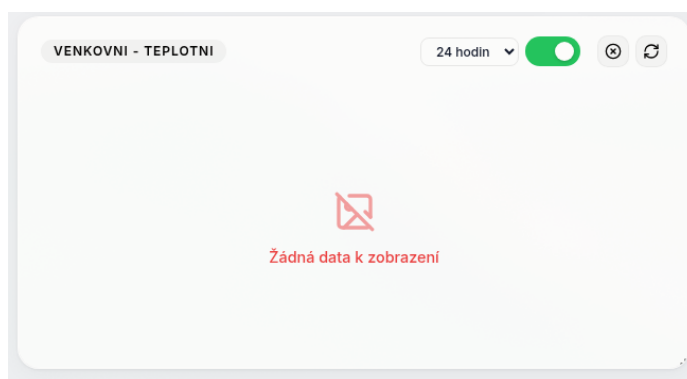
Pokud senzor neodpoví na dotaz po sběrnici Modbus RTU, knihovna `minimalmodbus` vyvolá výjimku po uplynutí nastaveného timeoutu (1 sekunda). Systém automaticky provede až tři opakované pokusy o čtení. Pokud ani jeden neuspěje, metoda `read_sensor` vrátí `None` a systém se pokusí načíst poslední známou hodnotu z databáze. Selhání je zalogováno a inkrementována statistika `failed_reads`.

Uživatel je o stavu senzoru informován prostřednictvím komponenty `Widget`, která při chybě místo dat vykreslí ikonu `ImageOff` s textovým popisem. Rozlišují se dva stavy: deaktivace senzoru

uživatel (zpráva „Senzor je neaktivní“, obrázek 6.16) a absence dat v databázi pro zvolený časový rozsah (zpráva „Žádná data k zobrazení“, obrázek 6.17).



Obrázek 6.16: Widget typu value s deaktivovaným senzorem a zprávou „Senzor je neaktivní“



Obrázek 6.17: Grafový widget s aktivním senzorem bez dostupných dat

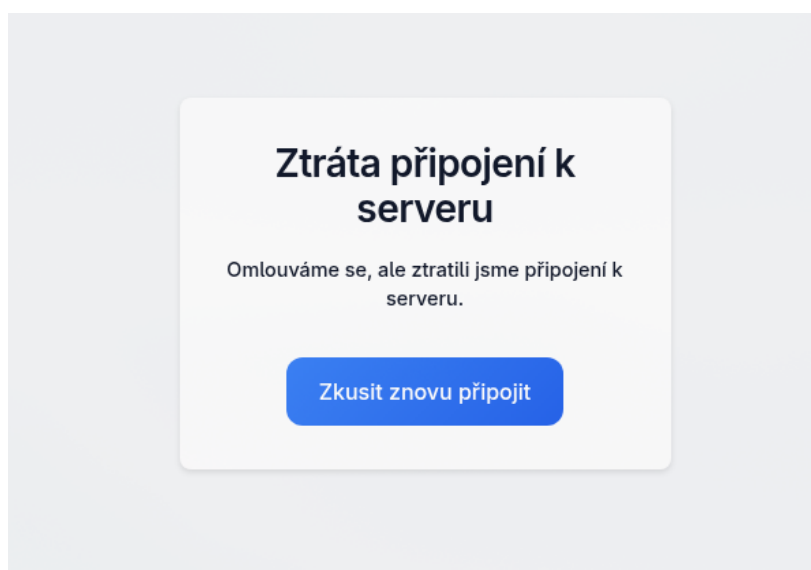
Výpadek serveru a zpracování na frontendu

Frontendová aplikace reaguje na výpadek komunikace se serverem na dvou úrovních. První úroveň je centrální modul `apiService.js`, ve kterém funkce `apiRequest` zpracovává všechny HTTP odpovědi. Sled zpracování chybové odpovědi probíhá následovně:

1. Pokud server vrátí stavový kód **500** nebo vyšší, funkce vyhodí výjimku typu `"server-error"`, která signalizuje nedostupnost serveru.
2. Pokud server vrátí kód **401**, spustí se pokus o automatickou obnovu access tokenu voláním endpointu `POST /auth/refresh-token`. Při úspěchu je původní požadavek automaticky zopakován s novým tokenem. Při neúspěchu je uživatel odhlášen a přesměrován na přihlašovací stránku.

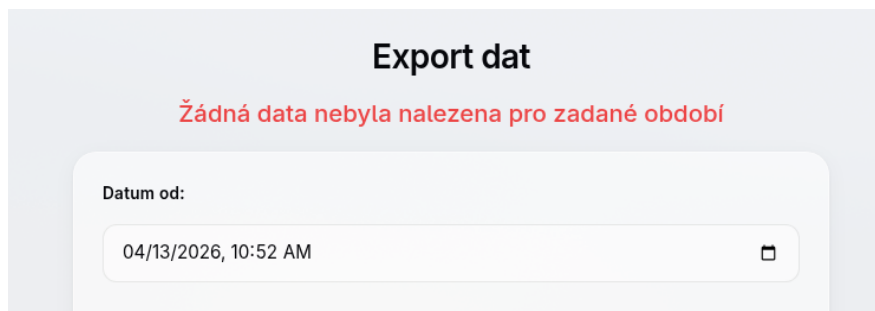
3. Pro ostatní chybové kódy (400, 403, 404) se z těla JSON odpovědi extrahuje zpráva z klíče `message` a předá volající komponentě, která ji zobrazí uživateli formou alertu.
4. Pokud požadavek selže kvůli timeoutu (`AbortError` po 15 sekundách) nebo síťové chybě (`NetworkError`), je vyvolána výjimka s popisem "`Server není dostupný`".

Druhou úrovní je komponenta `ProtectedRoute`, která při vykreslení ověří dostupnost serveru voláním `/auth/check`. Při nedostupnosti přesměruje na stránku `/server-error`, kde komponenta `ServerConnectionError` zobrazí informaci o výpadku a tlačítko pro opětovné připojení (obrázek 6.18). Po kliknutí se ověří token v `localStorage` a stav autentizace na backendu; uživatel je následně přesměrován do aplikace, nebo na přihlašovací stránku.



Obrázek 6.18: Stránka při výpadku komunikace se serverem

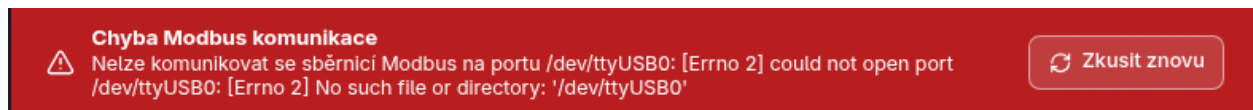
Jednotlivé stránky aplikace následně zobrazují chybové hlášky podle kontextu. Například stránka pro export dat zachytává výjimky z API volání a zobrazuje je textovým popisem chyby prostřednictvím elementu s třídou `error-message`, jak je ukázáno na obrázku 6.19. Postup zobrazení chyby je: backend vygeneruje JSON odpověď s chybovým kódem a zprávou, `apiService.js` tuto zprávu extrahuje a vyhodí jako výjimku, a volající komponenta ji zachytí v bloku `catch` a prezentuje uživateli.



Obrázek 6.19: Zobrazení chybové zprávy

6.6.1.2 Výpadek sběrnice RS-485

Pokud dojde k odpojení celé sběrnice RS-485, typicky vytažením převodníku USB–RS-485 z portu `/dev/ttyUSB0`, je tento stav ošetřen komponentou `ModbusStatusBanner`. Komponenta v pravidelném intervalu dotazuje stavový endpoint backendu a při zjištění nedostupnosti sběrnice zobrazí v horní části aplikace červený banner s popisem chyby a tlačítkem *Zkusit znovu*, kterým lze připojení obnovit bez restartu aplikace. Chybová hláška je zobrazena na obrázku 6.20.



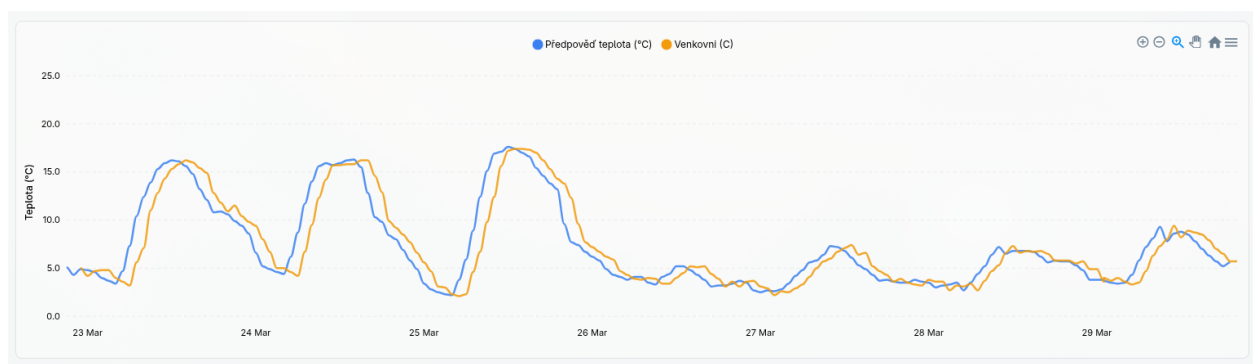
Obrázek 6.20: Chybová hláška komponenty `ModbusStatusBanner` při odpojení sběrnice RS-485

6.6.2 Porovnání naměřených dat

Pro ověření věrohodnosti naměřených dat bylo provedeno srovnání hodnot získaných ze senzorů systému s referenčními meteorologickými daty z veřejně dostupného API Open-Meteo. Toto API poskytuje historická i aktuální meteorologická data pro libovolnou lokalitu na základě zeměpisných souřadnic, a to bezplatně a bez nutnosti registrace.

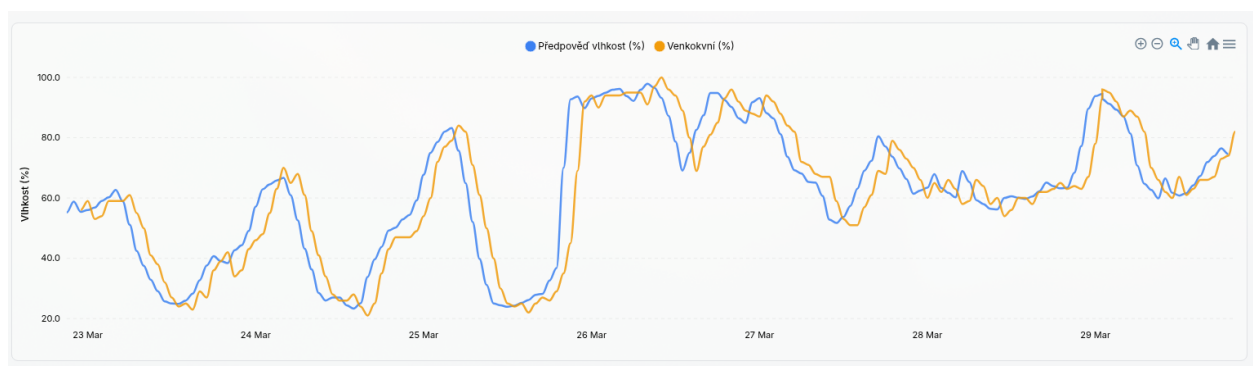
Sled porovnání probíhal následovně: ze systému byla exportována naměřená data teploty a vlhkosti ze senzoru SHT20 za vybrané časové období. Následně byla ze služby Open-Meteo stažena referenční data pro stejnou lokalitu a období. Obě datové sady byly srovnány z hlediska průběhu hodnot v čase, absolutních odchylek a celkových trendů.

Graf na obrázku 6.21 zobrazuje průběh hodinové teploty za období 23. 3. 2026 až 29. 3. 2026. Oranžová křivka představuje předpověď teploty z Open-Meteo API, zatímco modrá křivka odpovídá hodnotám naměřeným venkovním teplotním senzorem SHT20. Obě křivky vykazují velmi podobný denní průběh s výrazným střídáním nočních minim a denních maxim. Hodnoty se pohybovaly přibližně v rozsahu od 0 °C do 25 °C. Průběh obou křivek je úzce provázaný, přičemž lokální odchylky jsou způsobeny zejména rozdílnou polohou měřicího bodu senzoru oproti meteorologické stanici, z níž Open-Meteo čerpá data.



Obrázek 6.21: Graf s porovnáním dat z externí API s daty z teplotního senzoru

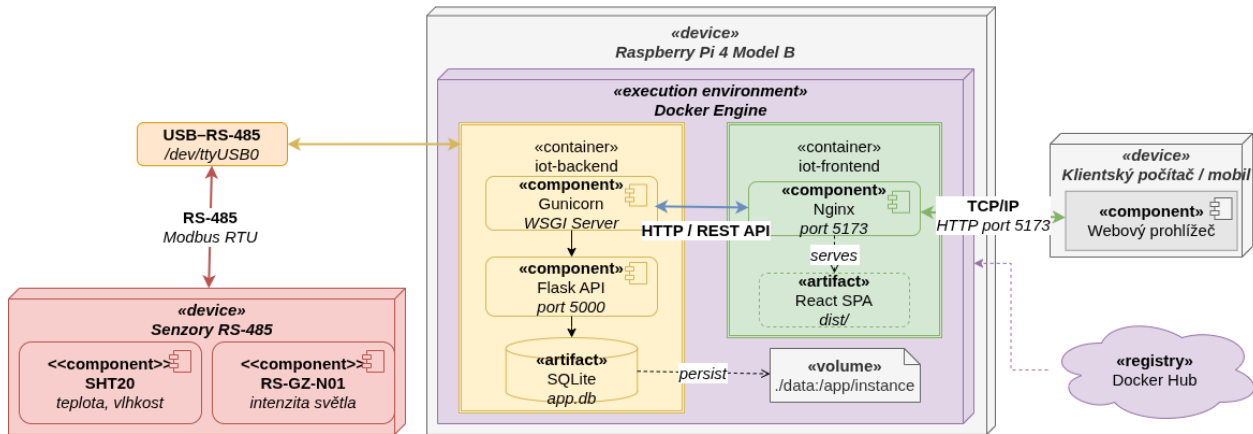
Graf na obrázku 6.22 zobrazuje hodinový průběh relativní vlhkosti vzduchu za totéž období. Modrá křivka znázorňuje předpověď relativní vlhkosti z Open-Meteo API a oranžová křivka odpovídá hodnotám naměřeným senzorem SHT20. Vlhkost se v průběhu sledovaného období pohybovala přibližně mezi 20 % a 100 %, přičemž obě datové sady sledují shodné trendy — zejména výrazný pokles vlhkosti v denních hodinách a její nárůst v noci. Větší absolutní odchylky oproti teplotnímu grafu jsou očekávatelné, neboť vlhkost je lokální.



Obrázek 6.22: Graf s porovnáním dat z externí API s daty z vlhkostního senzoru

6.7 Nasazení aplikace

Systém je nasazován pomocí nástroje Docker Compose, který orchestruje dva kontejnery: backend (Flask + Gunicorn) a frontend (Nginx). Předpřipravené Docker obrazy jsou publikovány na platformě Docker Hub, díky čemuž nasazení nevyžaduje přítomnost zdrojového kódu ani sestavovacích nástrojů na cílovém zařízení. Diagram nasazení je vidět na obrázku 6.23.



Obrázek 6.23: Diagram nasazení systému

6.7.1 Předpoklady

Pro nasazení na Raspberry Pi je vyžadován operační systém Raspberry Pi OS (64-bit), nainstalovaný Docker a připojený převodník USB–RS-485. Pro vývoj na běžném počítači postačuje libovolný operační systém s Dockerem. Převodník ani senzory nejsou v tomto režimu vyžadovány.

6.7.2 Spuštění systému

Repozitář aplikace je dostupný na adrese https://github.com/martinRihak/Bakalarska_prace, kde se v kořenovém adresáři nachází automatizovaný skript `install.sh`. Skript ověří přítomnost nástroje Docker, sestaví obrazy backendu a frontendu z příložených souborů `Dockerfile` (`iot-backend:local` a `iot-frontend:local`), vytvoří soubor `docker-compose.yml` a spustí kontejnery. Skript automaticky detekuje přítomnost převodníku USB–RS-485 a podle toho přizpůsobí konfiguraci – na Raspberry Pi mapuje zařízení `/dev/ttyUSB0` do kontejneru, na běžném počítači toto mapování vynechá. Nasazení vyžaduje pouze následující příkazy:

```
git clone https://github.com/martinRihak/Bakalarska_prace.git
cd Bakalarska_prace
chmod +x install.sh && ./install.sh
```

Listing 6.3: Klonování repozitáře a spuštění instalačního skriptu

Po dokončení skriptu je webové rozhraní dostupné na adrese `http://raspberrypi.local:5173`. Skript současně vytvoří adresář `data`, do kterého je připojena SQLite databáze běžící uvnitř backendového kontejneru, čímž je zajištěna perzistence dat mezi restartami. Kontejnery lze následně spravovat standardními příkazy Docker Compose (`docker compose ps`, `logs`, `down`). Detailnější popis spuštění aplikace a řešení chyb se nachází v souboru `README.md`.

6.7.3 Vytvoření administrátorského účtu

Pro vytvoření administrátorského účtu je v repozitáři připraven pomocný skript `create_admin.sh`, který vloží do databáze záznam s rolí `admin`. Skript nejprve ověří, zda je SQLite databáze zapisovatelná z hostitelského systému, a pokud ano, použije nástroj `sqlite3`. V opačném případě provede zápis prostřednictvím spuštěného backendového kontejneru pomocí příkazu `docker compose exec`.

```
chmod +x create_admin.sh && ./create_admin.sh
```

Listing 6.4: Vytvoření administrátorského účtu

Skript vytvoří účet s výchozími přihlašovacími údaji `username=admin` a `password=admin`. Z bezpečnostních důvodů je doporučeno změnit heslo bezprostředně po prvním přihlášení.

Kapitola 7

Závěr

Cílem této bakalářské práce bylo navrhnout a implementovat webovou aplikaci, která umožní správu a vizualizaci dat z připojených senzorů prostřednictvím platformy Raspberry Pi. Výsledkem je plně funkční systém, který komunikuje se senzory SHT20 a RS-GZ-N01 po sběrnici RS-485 s využitím protokolu Modbus RTU, ukládá naměřená data do lokální databáze SQLite a zprostředkovává je uživateli prostřednictvím interaktivního webového rozhraní. Backend byl implementován ve frameworku Flask a je organizován do tří vrstev — HTTP rout, servisních tříd a databázových modelů. Frontend byl realizován jako jednostránková aplikace v knihovně React, která nabízí konfigurovatelný dashboard s widgety typu čárového grafu, plošného grafu a zobrazení aktuální hodnoty. Aplikace dále umožňuje export a import dat ve formátech CSV a JSON, porovnání naměřených hodnot s meteorologickými daty z API Open-Meteo a dvouúrovňovou správu uživatelů s rolemi *user* a *admin*. Všechny stanovené cíle bakalářské práce byly úspěšně splněny.

Při realizaci se objevila řada technických obtížností, které bylo třeba překonat. Zásadním problémem bylo omezení microSD karty Raspberry Pi, jejíž životnost je vyjádřena počtem zápisových cyklů, které bylo vyřešeno dvouúrovňovou mezipamětí kombinující paměťovou frontu a cyklický zásobník typu ring buffer s následným dávkovým zápisem do databáze. Dalším problémem byl paralelní přístup více vláken ke sdílenému sériovému portu, který byl ošetřen kombinací zámku konkrétního senzoru a globálního sériového zámku. Systém také musel korektně reagovat na výpadky komunikace — v takové situaci automaticky provede tři opakované pokusy o čtení a následně načte poslední známou hodnotu z databáze, čímž je zajištěno, že uživatel vždy obdrží data.

Systém byl navržen tak, aby byl jednoduše rozšiřitelný. Aktuálně podporuje komunikaci pouze prostřednictvím protokolu Modbus RTU, a proto je třeba systém rozšířit o další typy komunikace, jako jsou Modbus TCP, MQTT nebo HTTP. Dalším rozšířením by mohla být pokročilá analýza naměřených dat zahrnující detekci anomálií a notifikace uživatele při překročení nastavených limitů. V neposlední řadě by systém mohl být doplněn o další typy vizualizačních widgetů a implementaci Progressive Web App pro pohodlnější použití na mobilních zařízeních.

Literatura

1. ISIKDAG, Umit; ISIKDAG, Umit. Internet of Things: Single-board computers. *Enhanced Building Information Models: Using IoT Services and Integration Patterns*. 2015. [Online; přístup: 24. ledna 2025].
2. ITNETWORK. *Úvod do Raspberry Pi*. Dostupné také z: <https://www.itnetwork.cz/hardware-pc/raspberry-pi/uvod-do-raspberry-pi>. [Online; přístup: 19. prosince 2024].
3. RASPBERRY PI FOUNDATION. *Software for Raspberry Pi*. n.d. Dostupné také z: <https://www.raspberrypi.com/software/>. [Online; přístup: 19. prosince 2024].
4. FOUNDATION, Raspberry Pi. *Raspberry Pi 5 Product Brief*. 2023. Dostupné také z: <https://datasheets.raspberrypi.com/rpi5/raspberry-pi-5-product-brief.pdf>. Přístup: 7. února 2025.
5. RASPBERRY PI 5 REVIEW. *Raspberry Pi 5 Review*. 2023. Dostupné také z: <https://www.pcmag.com/reviews/raspberry-pi-5>. [Online; přístup: 7. února 2025].
6. WEBFLOW. *Front-end*. 2024. Dostupné také z: <https://webflow.com/glossary/front-end>. [Online; přístup: 27. prosince 2024].
7. MDN WEB DOCS. *HTML: HyperText Markup Language*. n.d. Dostupné také z: <https://developer.mozilla.org/en-US/docs/Web/HTML>. [Online; přístup: 24. ledna 2025].
8. MDN WEB DOCS. *CSS: Cascading Style Sheets*. n.d. Dostupné také z: <https://developer.mozilla.org/en-US/docs/Web/CSS>. [Online; přístup: 24. ledna 2025].
9. MOZILLA DEVELOPER NETWORK. *JavaScript*. 2025. Dostupné také z: <https://developer.mozilla.org/en-US/docs/Web/JavaScript>. [Online; přístup: 18. dubna 2026].
10. META PLATFORMS. *React – The library for web and native user interfaces*. 2025. Dostupné také z: <https://react.dev/>. [Online; přístup: 18. dubna 2026].
11. STACK OVERFLOW. *Stack Overflow Developer Survey 2024 – Web Frameworks and Technologies*. 2024. Dostupné také z: <https://survey.stackoverflow.co/2024/technology>. [Online; přístup: 18. dubna 2026].
12. GOOGLE. *Angular – The web development framework for building the future*. 2025. Dostupné také z: <https://angular.dev/>. [Online; přístup: 18. dubna 2026].

13. YOU, Evan; VUE.JS CONTRIBUTORS. *Vue.js – The Progressive JavaScript Framework*. 2025. Dostupné také z: <https://vuejs.org/>. [Online; přístup: 18. dubna 2026].
14. VITE TEAM. *Vite – Next Generation Frontend Tooling*. 2025. Dostupné také z: <https://vite.dev/>. [Online; přístup: 18. dubna 2026].
15. DOCS, MDN Web. *Introduction to Django*. n.d. Dostupné také z: https://developer.mozilla.org/en-US/docs/Learn_web_development/Extensions/Server-side/Django/Introduction. Online; přístup: 4. ledna 2025.
16. DOCUMENTATION, Spring Boot. *Spring Boot – Spring Framework*. n.d. Dostupné také z: <https://spring.io/projects/spring-boot>. Online; přístup: 4. ledna 2025.
17. OPENJS FOUNDATION. *Express – Node.js web application framework*. 2025. Dostupné také z: <https://expressjs.com/>. [Online; přístup: 18. dubna 2026].
18. COPPERWAITE, Matt; LEIFER, Charles. *Learning flask framework*. Packt Publishing Ltd, 2015. Online; přístup: 22. dubna 2025.
19. ORACLE. *What is a Database?* n.d. Dostupné také z: <https://www.oracle.com/cz/database/what-is-database/>. Online; přístup: 4. ledna 2025.
20. LOGROCKET. *Angular vs. React vs. Vue.js: Comparing performance*. 2025. Dostupné také z: <https://blog.logrocket.com/angular-vs-react-vs-vue-js-performance/>. [Online; přístup: 18. dubna 2026].
21. PHAM, Tuan Anh. *Best popular backend frameworks by performance of throughput benchmark comparison and ranking in 2025* [online]. DEV Community, 2025. [cit. 2026-04-28]. Dostupné z: <https://dev.to/tuananhpham/popular-backend-frameworks-performance-benchmark-1bkh>.
22. RASPBERRYTIPS. *Which Database Is Best For Raspberry Pi? (My Top 5)*. 2024. Dostupné také z: <https://raspberrytips.com/best-database-raspberry-pi/>. [Online; přístup: 18. dubna 2026].
23. THINGSBOARD, INC. *ThingsBoard Community Edition Documentation*. 2026. Dostupné také z: <https://thingsboard.io/docs/>. [Online; přístup: 18. dubna 2026].
24. GRAFANA LABS. *Grafana documentation*. 2026. Dostupné také z: <https://grafana.com/docs/grafana/latest/>. [Online; přístup: 18. dubna 2026].
25. OPEN HOME FOUNDATION. *Home Assistant - Open source home automation*. 2026. Dostupné také z: <https://www.home-assistant.io/>. [Online; přístup: 18. dubna 2026].
26. DOCUMENTATION, Node-RED. *Node-RED – Low-code programming for event-driven applications*. n.d. Dostupné také z: <https://nodered.org/>. Online; přístup: 4. ledna 2025.

27. SILVA, JOSÉ DE JESÚS CAMACHO AND MARINÑO-SALGUERO, JORGE AND HERRERA-TAPIA, JORGE. *A Thorough Review and Comparison of Commercial and Open-Source IoT Platforms for Smart City Applications*. 2024. Dostupné také z: <https://www.mdpi.com/2079-9292/13/8/1465>. [Online; přístup: 18. dubna 2026].
28. KUGELSTADT, Thomas. *The RS-485 Design Guide: A short compendium for robust data transmission design*. Texas Instruments: Dallas, TX, USA. 2008.
29. STUDIO, Seeed. *RS-485 Shield for Raspberry Pi*. n.d. Dostupné také z: https://wiki.seeedstudio.com/RS-485_Shield_for_Raspberry_Pi/. Přístup: 7. února 2025.
30. SEHRAWAT, Deepti; GILL, Nasib Singh. Smart sensors: Analysis of different types of IoT sensors. In: *2019 3rd International Conference on Trends in Electronics and Informatics (ICOEI)*. IEEE, 2019, s. 523–528.

Příloha A

Přehled REST endpointů

Metoda	Endpoint	Popis	Přístup
<i>Autentizace (/auth)</i>			
POST	/auth/login	Přihlášení, vrací JWT token	Veřejný
POST	/auth/register	Registrace nového uživatele	Veřejný
POST	/auth/logout	Odhlášení, vymazání session	User
POST	/auth/refresh-token	Obnovení přístupového tokenu	User
GET	/auth/status	Ověření stavu přihlášení	User
GET	/auth/user	Data přihlášeného uživatele	User
<i>Senzory (/sensors)</i>			
GET	/sensors/getSensorHistory/<id>	Historie měření senzoru	User
GET	/sensors/getSensorHistoryHourly/<id>	Hodinová historie měření senzoru	User
GET	/sensors/getLatestSensorData/<id>	Poslední naměřená hodnota	User
GET	/sensors/getSensors	Seznam senzorů uživatele	User
GET	/sensors/available	Dostupné senzory k přiřazení	User
GET	/sensors/all	Seznam všech senzorů	Admin
POST	/sensors/create	Vytvoření nového senzoru	Admin
POST	/sensors/add-to-user	Přiřazení senzoru k uživateli	User

Pokračuje na další straně

Pokračování tabulky REST endpointů

Metoda	Endpoint	Popis	Přístup
PATCH	/sensors/<id>	Úprava parametrů senzoru	User
PATCH	/sensors/<id>/toggle-active	Aktivace/deaktivace senzoru	User
PATCH	/sensors/import_data	Import dat senzoru	User
POST	/sensors/export_data	Export dat (CSV/JSON)	User
DELETE	/sensors/remove-from-user/<id>	Odebrání senzoru od uživatele	User
DELETE	/sensors/delete/<id>	Smazání senzoru ze systému	Admin
<i>Dashboard (/dashboard)</i>			
GET	/dashboard/userDashBoards	Dashboardy uživatele	User
GET	/dashboard/widgets/<id>	Widgety dashboardu	User
POST	/dashboard/create	Vytvoření dashboardu	User
POST	/dashboard/widget	Přidání widgetu	User
POST	/dashboard/dashboard/<id>/save_positions	Uložení pozic widgetů	User
DELETE	/dashboard/dashboard/<id>	Smazání dashboardu	User
<i>Widgety (/widget)</i>			
DELETE	/widget/delete/<d_id>/<w_id>	Odebrání widgetu	User
<i>Správa uživatelů (/users)</i>			
GET	/users	Seznam všech uživatelů	Admin
GET	/users/<id>	Detail uživatele	Admin
POST	/users	Vytvoření uživatele	Admin
PATCH	/users/<id>	Úprava uživatele	Admin
DELETE	/users/<id>	Smazání uživatele	Admin
GET	/users/<id>/sensors	Senzory uživatele	Admin
PATCH	/users/<id>/sensors/<s_id>	Úprava senzoru uživatele	Admin
POST	/users/<id>/sensors	Vytvoření senzoru pro uživatele	Admin
POST	/users/<id>/sensors/<s_id>	Přiřazení existujícího senzoru	Admin

Pokračuje na další straně

Pokračování tabulky REST endpointů

Metoda	Endpoint	Popis	Přístup
GET	<code>/users/me/location</code>	Uložená poloha uživatele	User
PATCH	<code>/users/me/location</code>	Uložení polohy uživatele	User
<i>Počasí (/weather)</i>			
GET	<code>/weather</code>	Předpověď počasí pro lokaci	User

Tabulka A.1: Přehled REST endpointů

Příloha B

Struktura systému

```
home-automation-system/
├── backend/ ..... Flask Python backend aplikace
│   ├── models/ ..... složka s databázovými modely (SQLAlchemy)
│   ├── routes/ ..... složka s definicemi API endpoint rout
│   ├── services/ ..... složka s business logikou aplikace
│   ├── utils/ ..... složka s pomocnými utilitami
│   ├── run.py ..... vstupní bod pro spuštění vývojového serveru
│   ├── wsgi.py ..... WSGI rozhraní pro produkční nasazení
│   └── Dockerfile ..... definice Docker obrazu backendu
├── data/ ..... složka pro datové soubory aplikace
├── frontend/ ..... React Vite frontend aplikace
│   ├── node_modules/ ..... nainstalované npm závislosti
│   └── src/ ..... zdrojový kód React aplikace
│       ├── api/ ..... moduly pro komunikaci s REST API backendu
│       ├── assets/ ..... statické soubory projektu
│       │   └── css/ ..... CSS soubory se styly aplikace
│       ├── components/ ..... znovupoužitelné React komponenty
│       │   ├── common/ ..... obecné sdílené komponenty
│       │   ├── forms/ ..... formulářové komponenty
│       │   ├── layout/ ..... komponenty pro rozvržení stránky
│       │   └── widgets/ ..... komponenty widgetů dashboardu
│       ├── features/ ..... komplexní funkční moduly aplikace
│       │   ├── auth/ ..... modul pro autentizaci uživatelů
│       │   │   └── Login/ ..... komponenty přihlašovací stránky
│       │   └── dashboard/ ..... modul pro správu dashboardu
│       └── hooks/ ..... vlastní React hooky
```

└─ pages/	komponenty jednotlivých stránek
└─ utils/	pomocné funkce a utilitní moduly
└─ index.html	hlavní HTML šablona aplikace
└─ vite.config.js	konfigurace Vite build nástroje
└─ Dockerfile	definice Docker obrazu frontendu
└─ docker-compose.yml	Docker Compose konfigurace pro nasazení
└─ install.sh	Instalační script
└─ create_admin.sh	Vytvoření admina
└─ BK_projekt.db	SQLite databáze projektu

Příloha C

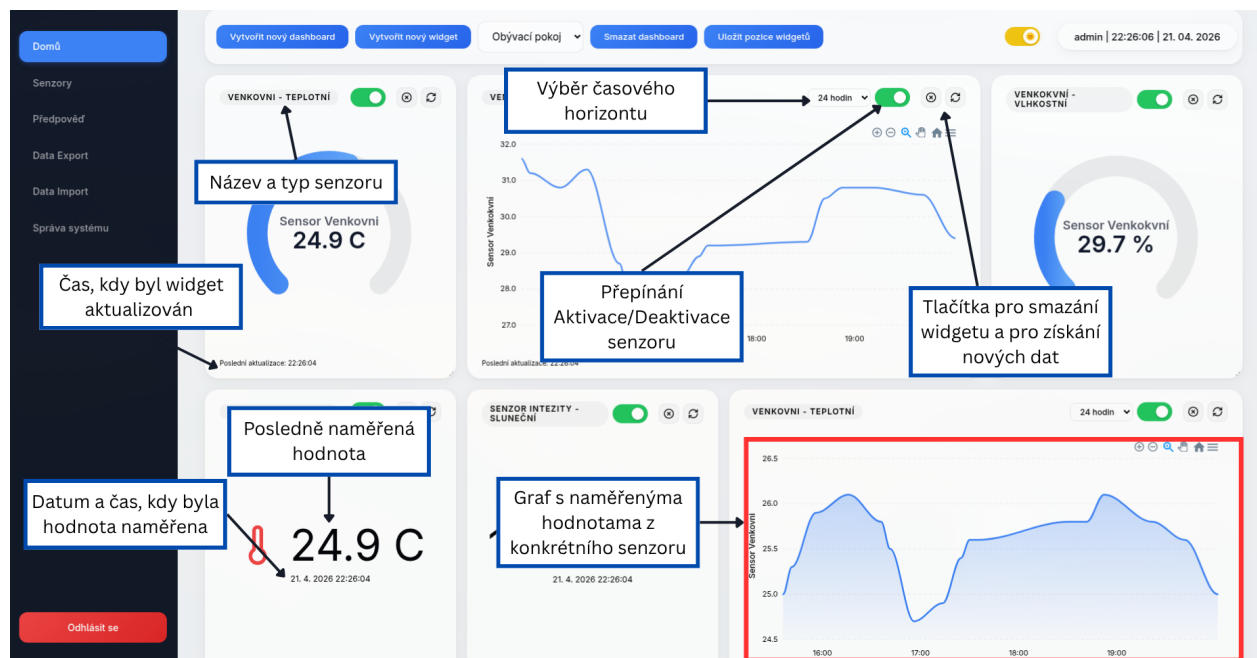
Dokumentace systému

C.1 Hlavní Frontend systému

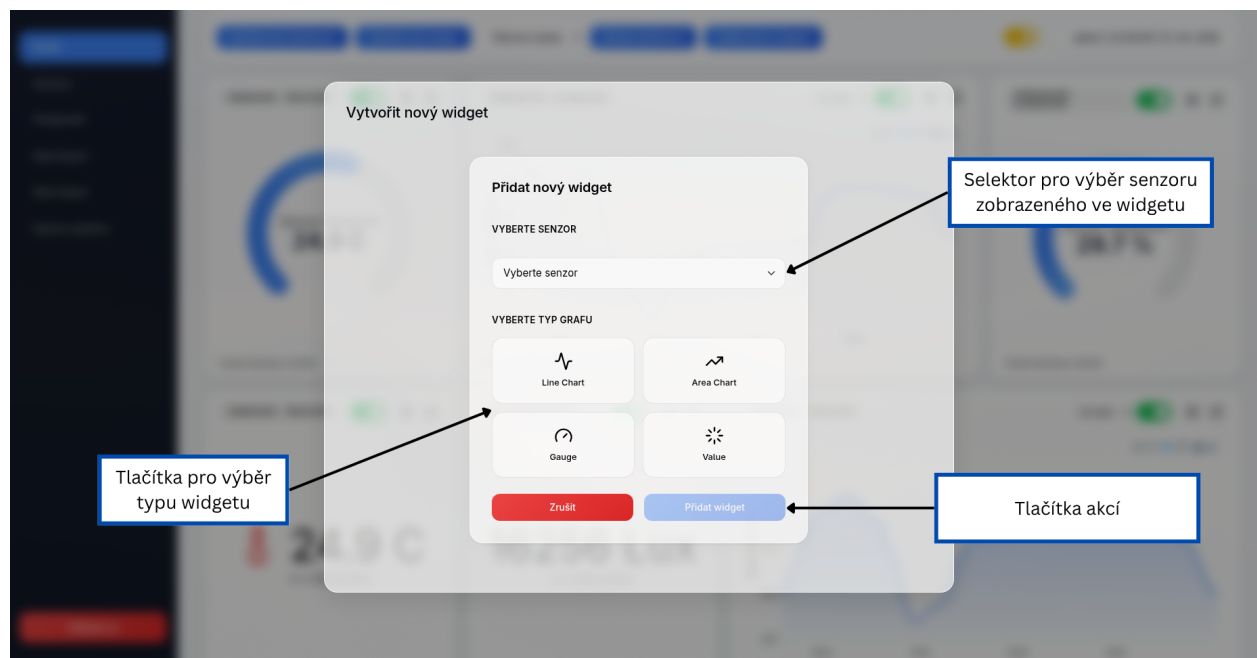
C.1.1 Základní popis hlavního dashboardu



C.1.2 Detailní popis hlavního dashboardu



C.1.3 Rozhraní pro vytvoření nového widgetu



C.1.4 Senzory uživatele

The screenshot shows the 'Moje senzory' page. A sidebar on the left contains links: 'Senzory', 'Předpověď', 'Data Export', 'Data Import', and 'Správa systému'. The main area has a 'Přidat senzor' button at the top left. Below it is a table with three sensors. Annotations with arrows point to specific elements: 'Tlačítko pro přidání nového senzoru' points to the 'Přidat senzor' button; 'Základní informace o senzoru' points to the first three columns of the table; 'Přepínání aktivity' points to the toggle switch in the 'AKTIVNÍ' column; and 'Tlačítko pro smazání a úpravu daného senzoru.' points to the 'Upravit' and 'Smazat' buttons in the 'AKCE' column.

Tlačítko pro přidání nového senzoru

Moje senzory

NÁZEV	TYP	JEDNOTKA	AKTIVNÍ	AKCE
Venkovní	Teplotní	C	☒	Upravit Smazat
Venkovní	Vlhkostní	%	☒	Upravit Smazat
Senzor intezity	Sluneční	Lux	☒	Upravit Smazat

Základní informace o senzoru

Přepínání aktivity

Tlačítko pro smazání a úpravu daného senzoru.

Odhlásit se

C.1.5 Rozhraní pro přidání senzoru

The screenshot shows the 'Přidat senzor' modal form. It contains three buttons: 'Přidat existující', 'Vytvořit nový', and 'Zrušit'. Annotations with arrows point to these buttons: 'UI pro přidání senzoru' points to the modal form; 'Přidání již existujícího senzoru v systému' points to the 'Přidat existující' button; 'Vytvoření nového senzoru do systému' points to the 'Vytvořit nový' button; and 'Zrušit akci přidávání senzoru' points to the 'Zrušit' button.

UI pro přidání senzoru

Přidat senzor

[Přidat existující](#) [Vytvořit nový](#) [Zrušit](#)

Přidání již existujícího senzoru v systému

Vytvoření nového senzoru do systému

Zrušit akci přidávání senzoru

C.1.6 Formulář pro úpravu senzoru

The form 'Upravit senzor' contains the following fields and controls, each with an explanatory annotation:

- Název nového senzoru**: Points to the 'NÁZEV' text input field.
- Na jaké adrese se senzor nachází**: Points to the 'ADRESA' text input field.
- Přenosová rychlost sériové linky pro daný senzor**: Points to the 'FUNCTION CODE' dropdown menu.
- Jednotka naměřených hodnot**: Points to the 'JEDNOTKA' dropdown menu.
- V jakém časovém intervalu má systém číst data ze senzoru**: Points to the 'SAMPLING RATE (MINUTY)' dropdown menu.
- O jaký typ senzoru se jedná**: Points to the 'TYP SENZORU' dropdown menu.
- Definuje, zda je senzor/zařízení read-only nebo read/write**: Points to the 'BIT' dropdown menu.
- Kolika desetinnými čísly má systém vydělit hodnoty ze senzoru pro správný výsledek**: Points to the 'SCALING' dropdown menu.
- Maximální a minimální možná naměřená hodnota senzorem**: Points to the 'MINIMÁLNÍ HODNOTA' and 'MAXIMÁLNÍ HODNOTA' text input fields.
- Tlačítka pro zrušení akce nebo pro uložení nového senzoru**: Points to the 'Zrušit' (red) and 'Uložit' (blue) buttons at the bottom.

C.1.7 Vyhledávání lokality pro předpověď



C.1.8 Výsledky předpovědi a porovnání senzorů



C.1.9 Export naměřených dat

Export dat

Datum od: mm/dd/yyyy, --:-- --

Datum do: mm/dd/yyyy, --:-- --

Výběr senzorů:

- ☐ Venkovní (C)
- ☐ Venkovní (%)
- ☐ Senzor Intezity (Lux)

Název souboru (nepovinné):
Pokud nevyplníte, použije se výchozí název

Formát exportu: JSON

Exportovat data

Výběr rozmezí exportovaných dat

Výběr senzorů, kterým se vyexportují data do souboru

Pojmenování exporovaného souboru

Tlačítko pro exportování dat

Výběr formátu exportovaného souboru

C.1.10 Import dat do vybraného senzoru

The screenshot shows the 'Import dat' form in the system. The left sidebar contains links: Domů, Senzory, Předpověď, Data Export, Data Import (highlighted), and Správa systému. The form has a dropdown for 'Cílový senzor:' with the text '— Vyberte senzor —'. Below it is a section 'Soubor (CSV nebo JSON):' with a dashed box containing an upload icon and the text 'Přetáhněte soubor nebo klikněte pro výběr' and 'json · csv'. A blue button 'Importovat data' is at the bottom. Annotations with arrows point to: 'Výběr senzoru, do kterého se importovaná data načtou' (pointing to the dropdown), 'Nahrání souboru v .json a .csv formátu' (pointing to the upload area), and 'Tlačítko pro Importování dat' (pointing to the 'Importovat data' button).

admin | 22:32:03 | 21. 04. 2026

C.1.11 Administrátorská stránka

The screenshot shows the administrative page with two tables. The top table is 'Přehled všech uživatelů v systému' and the bottom table is 'Všechny senzory v systému'. Annotations with arrows point to: 'Přehled všech uživatelů v systému' (pointing to the top table), 'Tlačítko pro vytvoření nového uživatele' (pointing to the 'Vytvořit uživatele' button), 'Přehled všech senzorů konkrétního uživatele' (pointing to the first table), 'Tlačítka pro přiřazení nového senzoru uživateli nebo již existujícího' (pointing to 'Vytvořit nový senzor' and 'Přidat existující senzor' buttons), 'Tlačítka pro úpravu nebo smazání konkrétního senzoru nebo uživatele' (pointing to 'Upravit' and 'Smazat' buttons), and 'Přehled všech senzorů v systému' (pointing to the bottom table).

admin | 22:32:21 | 21. 04. 2026

ID	UŽIVATELSKÉ JMÉNO	EMAIL	ROLE	VYTVOREN	POSLEDNÍ PŘIHLÁŠENÍ	AKCE
1	admin	admin@admin.cz	admin	4/14/2026, 7:14:45 PM	4/21/2026, 8:32:15 PM	<button>Upravit</button> <button>Smazat</button>
2	Adam	adam@adam.cz	user	4/21/2026, 3:40:19 PM	-	<button>Upravit</button> <button>Smazat</button>

NÁZEV	TIPOVÝ	JEDNOTKA	ADRESA	SAMPLING	AKTIVNÍ	AKCE
Pokoj 1313B	Teplotní	°C	10	Ne	<button>Upravit</button> <button>Smazat</button>	
Pokoj 1313B	Vlhkostní	%	10	Ano	<button>Upravit</button> <button>Smazat</button>	

NÁZEV	TYP	JEDNOTKA	ADRESA	SAMPLING	AKTIVNÍ	AKCE
Venkovní	Teplotní	C	1	10	Ano	<button>Upravit</button> <button>Smazat</button>
Venkovní	Vlhkostní	%	2	10	Ano	<button>Upravit</button> <button>Smazat</button>
Senzor Intezity	Sluneční	Lux	64	10	Ano	<button>Upravit</button> <button>Smazat</button>
Pokoj 1313B	Teplotní	°C	1	10	Ne	<button>Upravit</button> <button>Smazat</button>
Pokoj 1313B	Vlhkostní	%	2	10	Ano	<button>Upravit</button> <button>Smazat</button>

C.1.12 Formulář pro přidání nového uživatele

The screenshot shows a web application interface with a dark sidebar on the left containing links: Domů, Senzory, Předpověď, Data Export, Data Import, and Správa systému. The main content area features a form titled 'Přidat nového uživatele' enclosed in a red border. The form includes a 'Zpět' button at the top, followed by input fields for 'UŽIVATELSKÉ JMÉNO *', 'HESLO *', and 'EMAIL *'. Below these is a 'ROLE' dropdown menu currently set to 'Uživatel', and a 'Vytvořit uživatele' button at the bottom. Annotations with arrows point to various elements: 'emailová adresa nového uživatele' points to the email field; 'Výběr role nového uživatele' points to the role dropdown; 'Tlačítko pro vytvoření nového uživatele' points to the 'Vytvořit uživatele' button; 'Formulář pro přidání nového uživatele z administrátorské stránky' points to the form title; 'Tlačítko pro návrat do administrátorské stránky' points to the 'Zpět' button; 'Pojmenování nového uživatele' points to the username field; and 'Heslo pro nového uživatele' points to the password field.

C.1.13 Formulář pro registraci uživatele

The screenshot shows a web application interface with a light gray background. The main content area features a form titled 'Registrace' enclosed in a red border. The form includes input fields for 'Uživatelské jméno', 'E-mail', 'Heslo', and 'Potvrzení hesla'. Below these is a 'Zaregistrovat se' button and a link 'Máte již účet? Přihlaste se'. Annotations with arrows point to various elements: 'Uživatelské jméno uživatele pro přihlášení' points to the username field; 'Email uživatele' points to the email field; 'Heslo uživatele a jeho zopakování' points to the password and confirmation fields; 'Tlačítko pro registrování uživatele' points to the 'Zaregistrovat se' button; 'Odkaz na přihlašovací stránku do systému' points to the 'Máte již účet? Přihlaste se' link; and 'Formulář pro registraci uživatele s rolí uživatel' points to the form title.

C.1.14 Formulář pro přihlášení uživatele

The diagram illustrates a user login form titled "Přihlášení". The form is enclosed in a red border and contains the following elements:

- Uživatelské jméno**: A text input field for the user's username.
- Heslo**: A text input field for the user's password.
- Přihlásit se**: A blue button to submit the login information.
- Nemáte účet? Zaregistrujte se**: A blue link for users who do not have an account.

Five external labels with arrows point to specific parts of the form:

- Uživatelské jméno uživatele pro přihlášení**: Points to the username input field.
- Formulář pro přihlášení uživatele**: Points to the entire form area.
- Uživatelské heslo pro přihlášení**: Points to the password input field.
- Tlačítko pro přihlášení uživatele**: Points to the "Přihlásit se" button.
- Odkaz na registrační stránku do systému**: Points to the "Nemáte účet? Zaregistrujte se" link.